Homework 3: Implementation of Linked List, Recursion, Sorting & CPU/Disk Consumption

CS515-10, Fall 2025
Purdue University Northwest
09/26/2025

Name	Email	Contributions
Sachin Kumar	**kumar957@pnw.edu**	**Self**

Table of Contents

Table of Figures

ABSTRACT

This task aims on the design and execution of toy “memory database” using linked list data structures to efficiently manage student records stored in CSV file. This memory database is developed using any of the two different programming languages from Julia, Python, Java, GO, and C/C++. Basically, this memory database must showcase the three core recursive operations: fetch all the records from student-data.csv file and load into a linked list in-memory; ensure that memory database should adapt changes after the addition or removal of record; export all the in-memory records to a new CSV file; applied bubble and insertion sorting algorithm to the in-memory data; and exported back the sorted data to a CSV files through recursive functions. The work also analyzes the CPU and disk usage for both sorting algorithms on Ubuntu, highlighting the performance differences. This technique illustrates the importance of singly and doubly linked list data structures, indicates node traversal, memory allocation, sorting methods, and system resource profiling. It also provides hands-on exposure with recursion and dynamic data management in a Linux-based environment.

I. INTRODUCTION

Efficient data management is a big challenge for data engineers and software developers. The goal of this task is to store & manipulate the data efficiently stored in a memory database specially when the data is subjected to frequent changes. Memory database is not a disk-based database like Oracle SQL or MySQL, it's basically da program that stores data in RAM using linked list along with recursive functions to load the data & export to a CSV file.

Linked list solves this problem due to its logical properties and provides dynamic solution of handling the data by enabling memory allocation. In addition to this, linked list provides consistent insertion as well as deletion of elements without shifting large blocks of elements.

Using bubble sort and insertion sort algorithm, the implementation to sort the data based on any chosen column, handling both numeric and string types. The sorted data is exported using a recursive CSV export function. This study also measures system performance (CPU and disk usage) to compare the computational performance of the two sorting methods.

This project was executed in Python programming by applying doubly linked list approach and Julia programming by applying singly linked list approach within an Ubuntu 25.04 installed on Oracle VirtualBox.

II. REQUIREMENTS

1. Environment Setup

- **Host Machine:** Windows 11
- **Virtualization Tool:** Oracle VirtualBox 7.1.12 (Click here to download)
- **Guest OS:** Ubuntu 25.04 (Click here to download)
- **IDE:** Visual Studio Code
- **Preferred Programming Language:**
    - Python
    - Julia
- **Required Libraries:**
    - **Python:** csv, os, time
    - **Julia:** CSV, DataFrames, Dates

2. Dataset

A dataset is a collection of records and data used for analysis, research or training models. It can be structured, unstructured or semi-structured, depending upon its requirement. For this task, a structured records of students (student-data.csv) is used that can be accessed from kaggle portal to implement the insertion & removal of records using linked list concept.

III. INSTALL VIRTUAL MACHINE ON WINDOWS 11

1. Download the setup file of Oracle VirtualBox - 7.1.12 by clicking here.

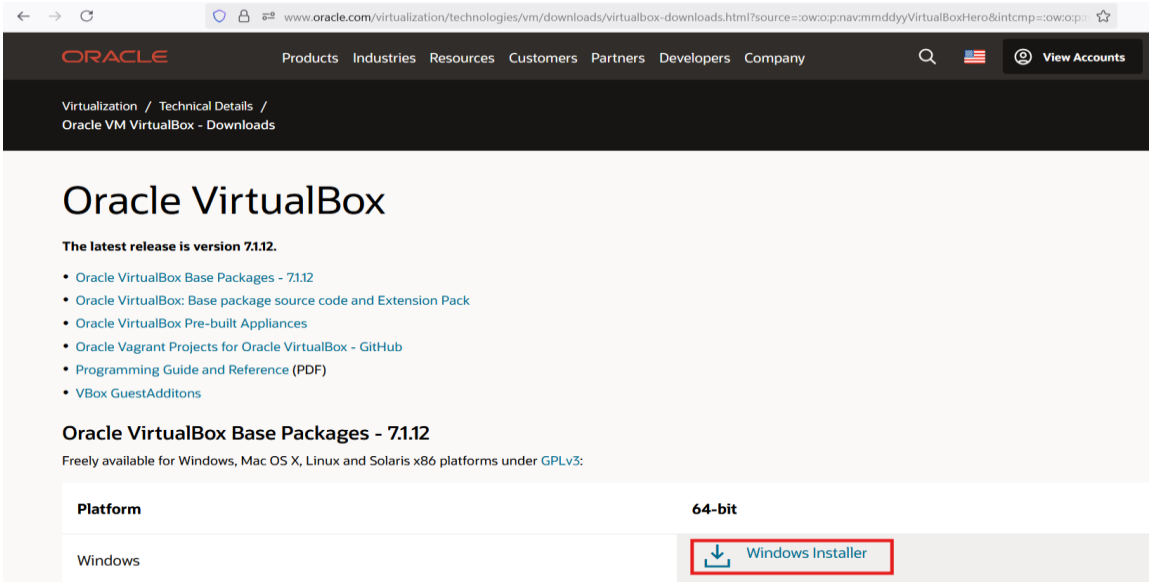


Figure 1 – Portal to download Oracle VirtualBox – 7.1.12

2. Locate the downloaded file in download folder & double click on installer (.exe file) to launch the setup wizard.

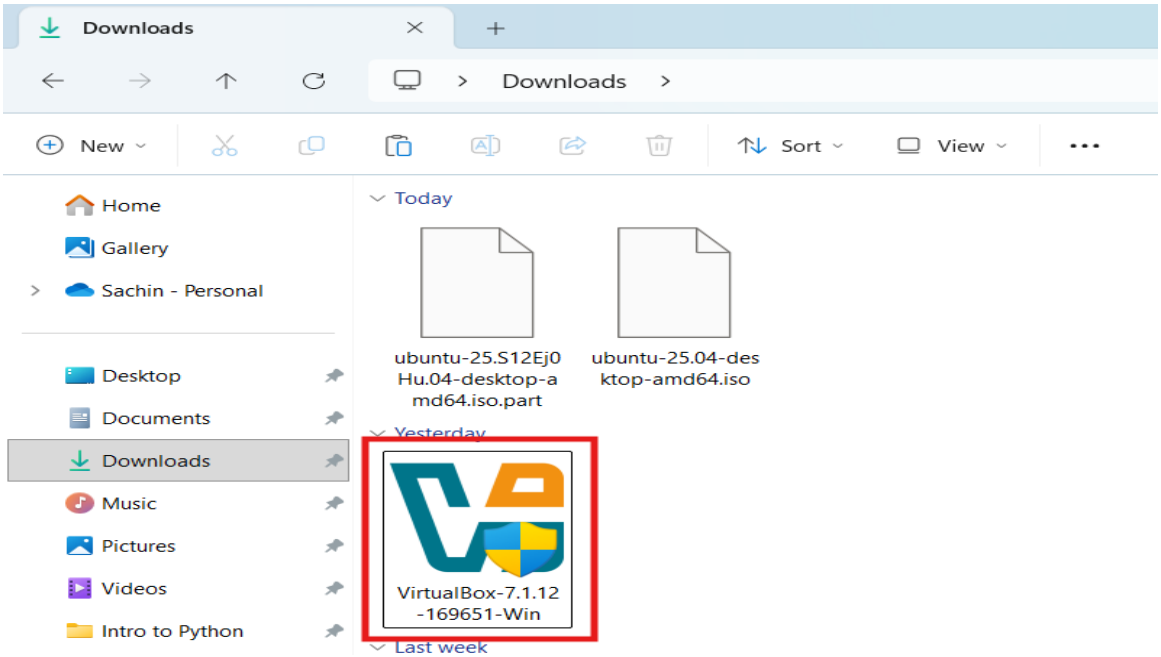


Figure 2 – Download folder where VirtualBox Setup file is downloaded

3. **If prompted by User Account Control (UAC), click Yes to allow installation.**

4. **In the welcome setup wizard, click next & choose the installation location (default is recommended).**

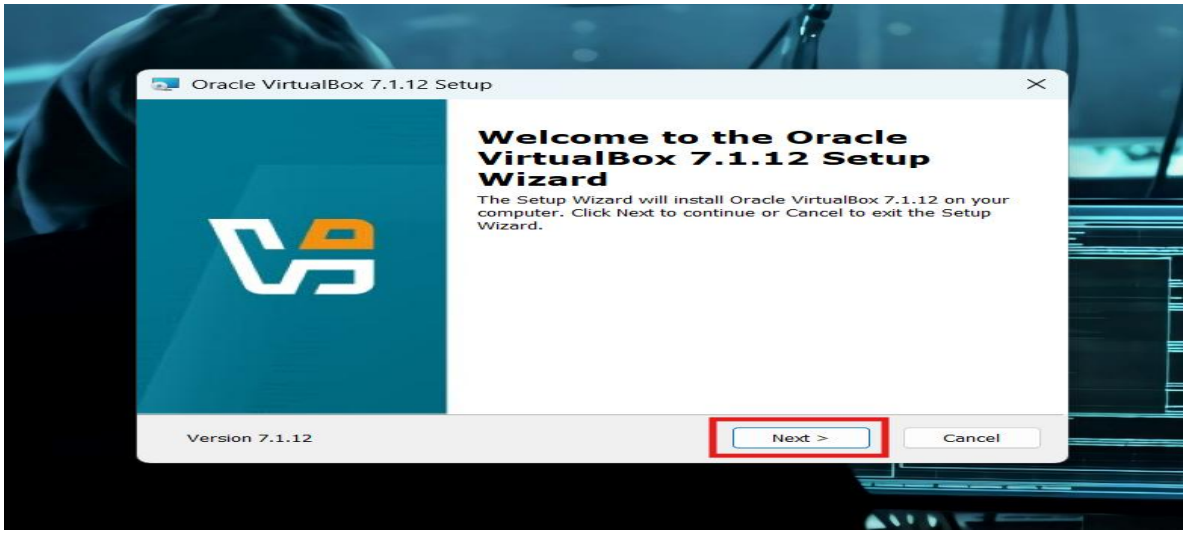


Figure 3 – Oracle VirtualBox 7.1.12 Setup Wizard

5. **Accept the terms in the License Agreement & click next.**

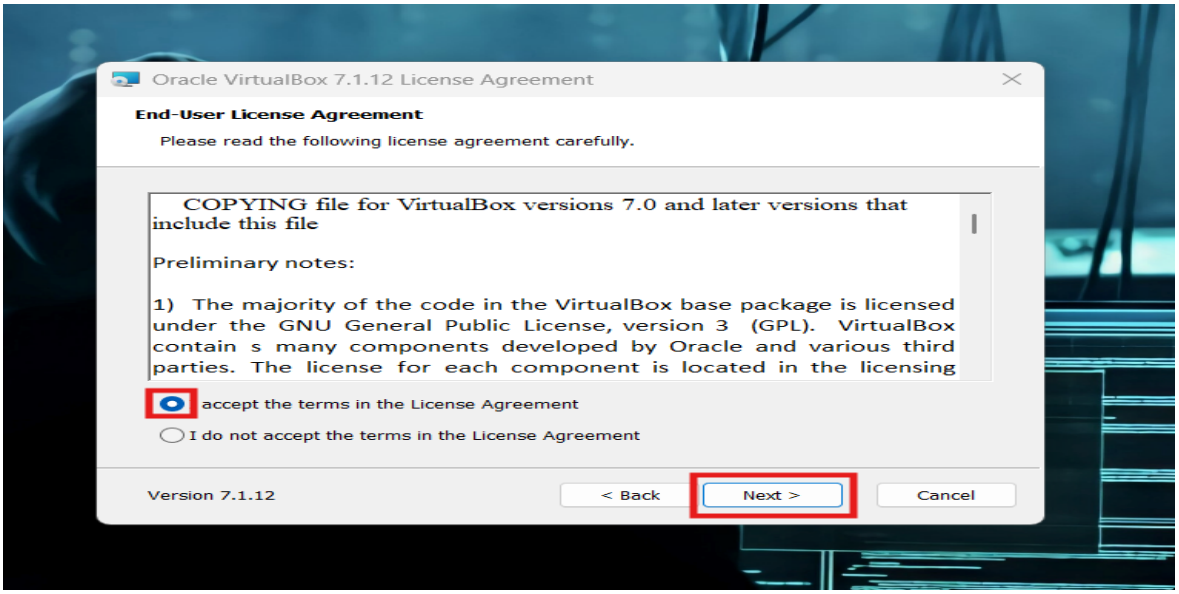


Figure 4 – End User License Agreement to install VirtualBox

6. Select components to install (leave default checked) & click next.

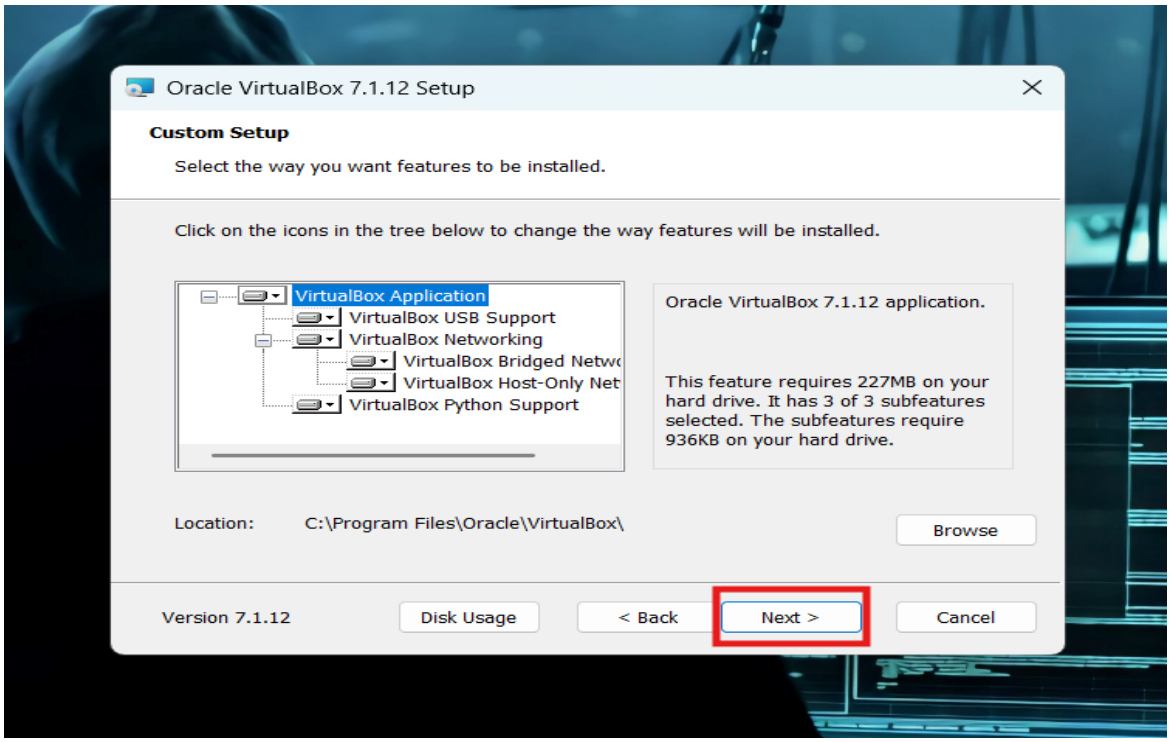


Figure 5 – Option to select required components of VirtualBox for installation

7. A screen of warning with network interfaces reset will be displayed, click yes.

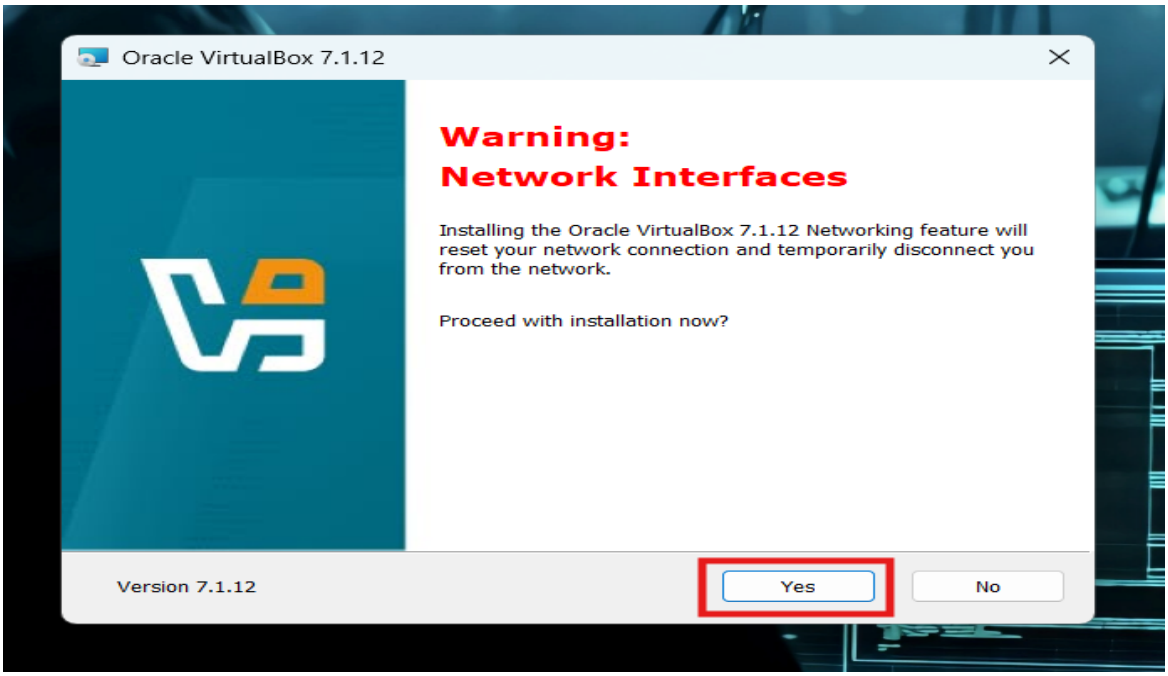


Figure 6 – A warning for Network Interfaces Reset while installing VirtualBox

8. Choose whether to create a Desktop Shortcut and Start Menu entries (set as default), click next.

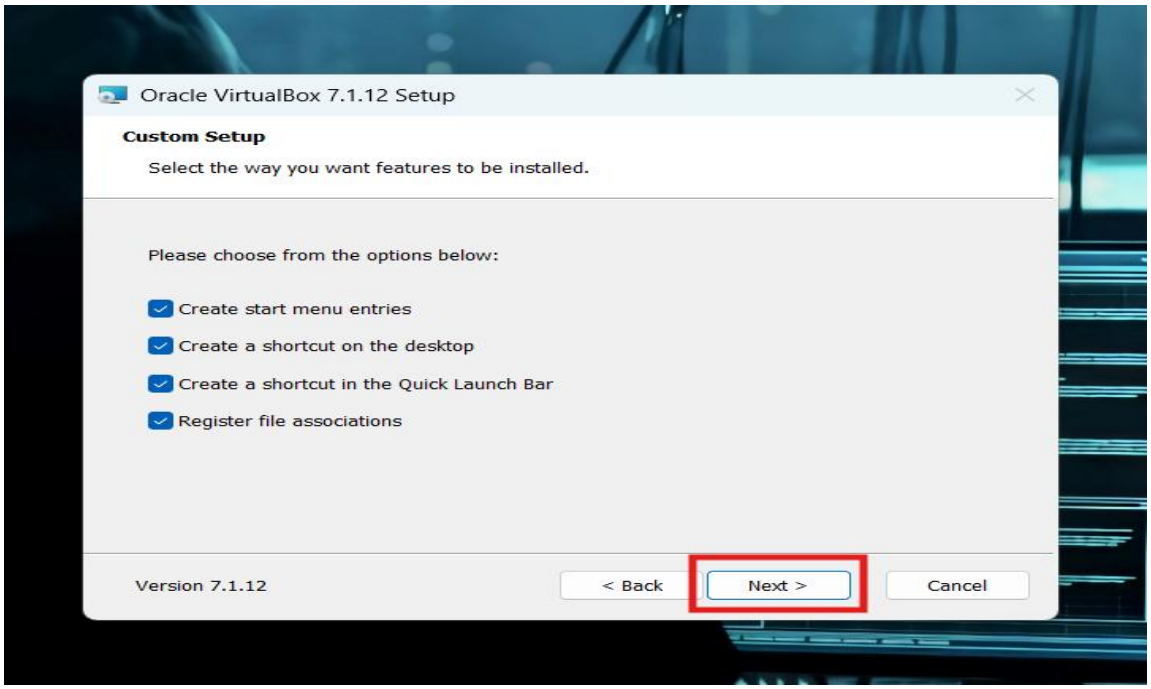


Figure 7 – Custom Setup options while Installing VirtualBox

9. Click install to begin installation.

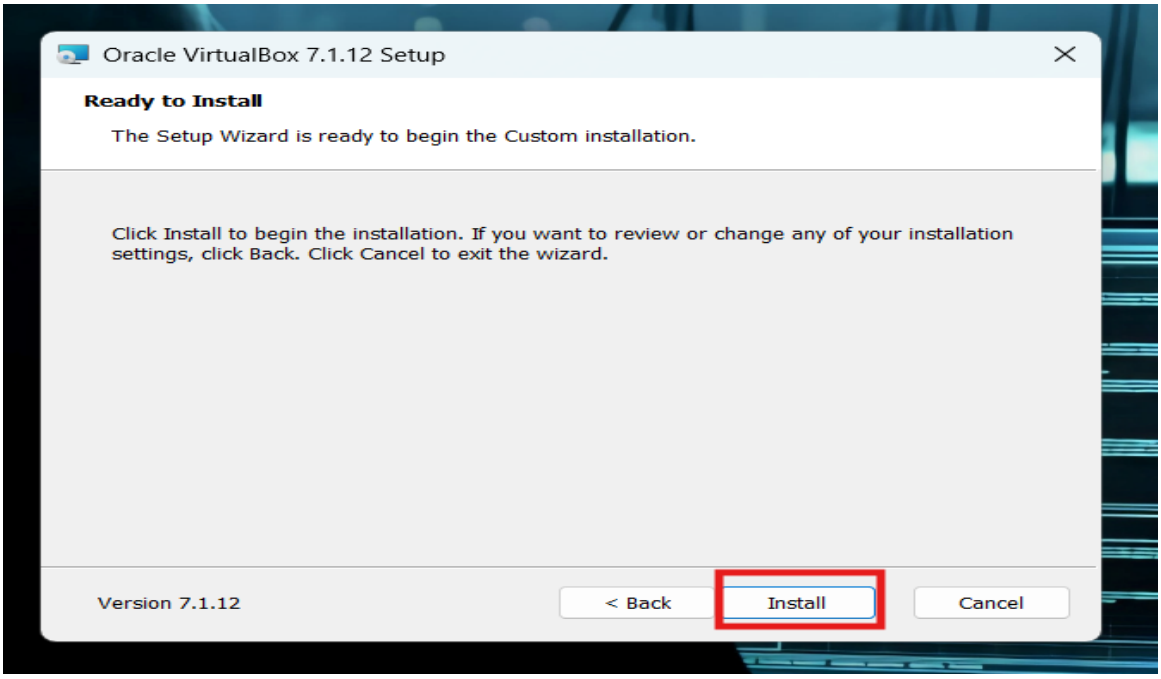


Figure 8 – Ready for the installation of VirtualBox

10. Once installation completes, click finish & ensure Start Oracle VirtualBox after installation is checked.

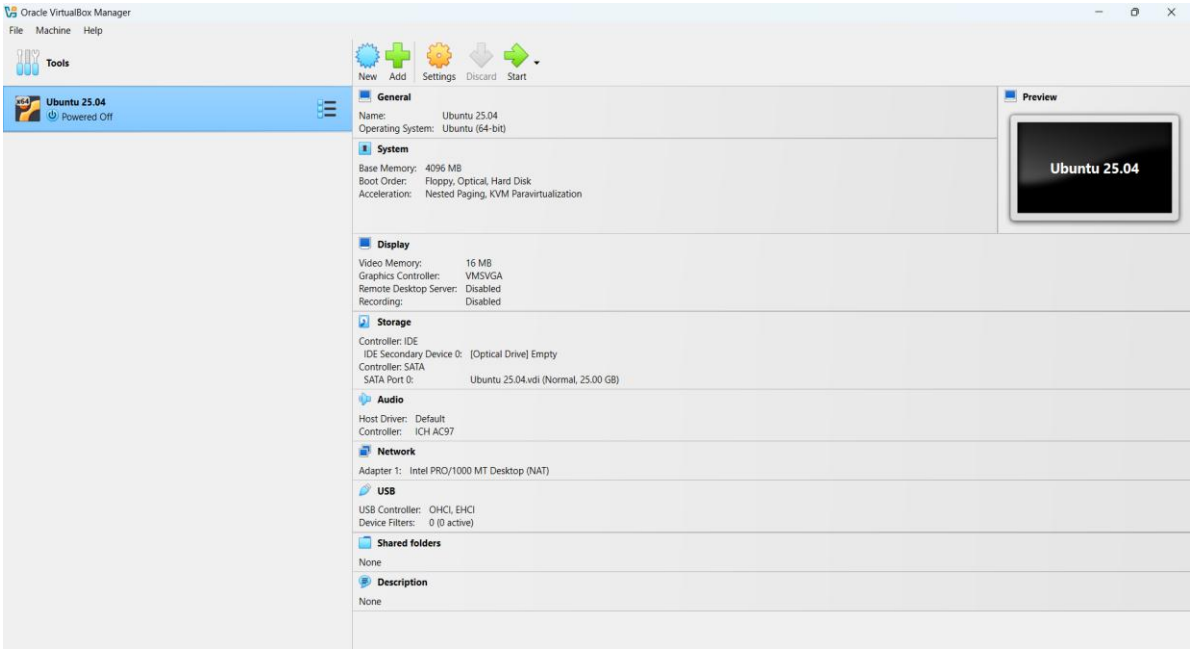


Figure 9 – Installed setup of an Oracle VirtualBox 7.1.12

IV. INSTALL UBUNTU LINUX ON VIRTUAL MACHINE

1. Download the Ubuntu 25.04 Desktop ISO file by clicking here.

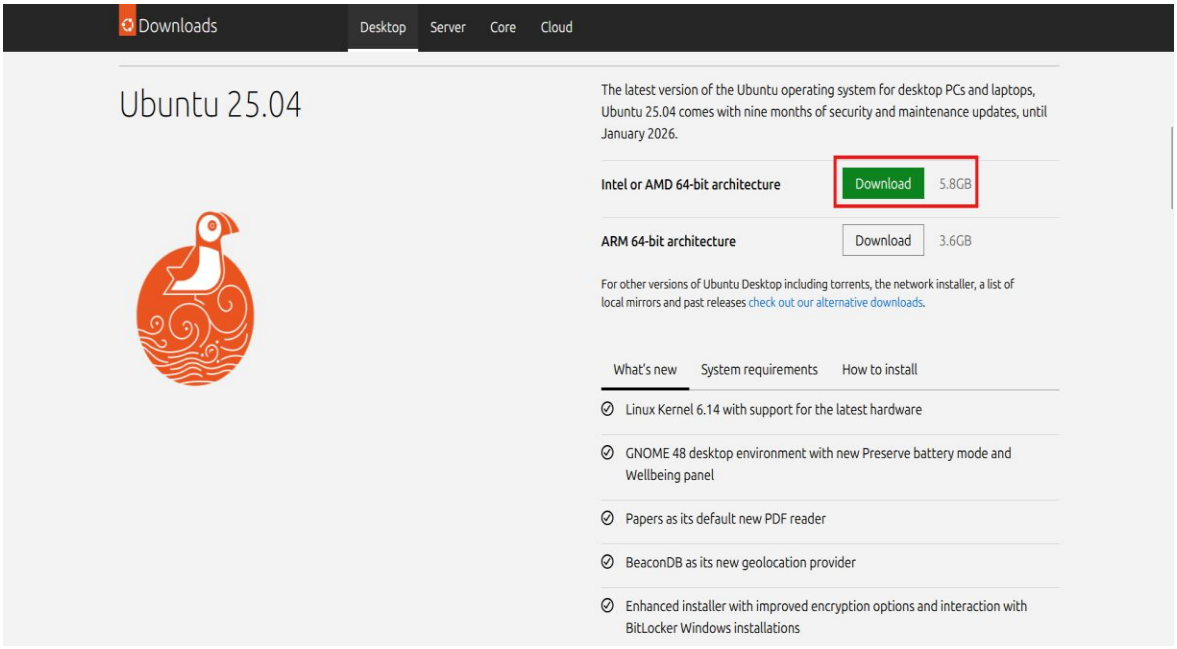
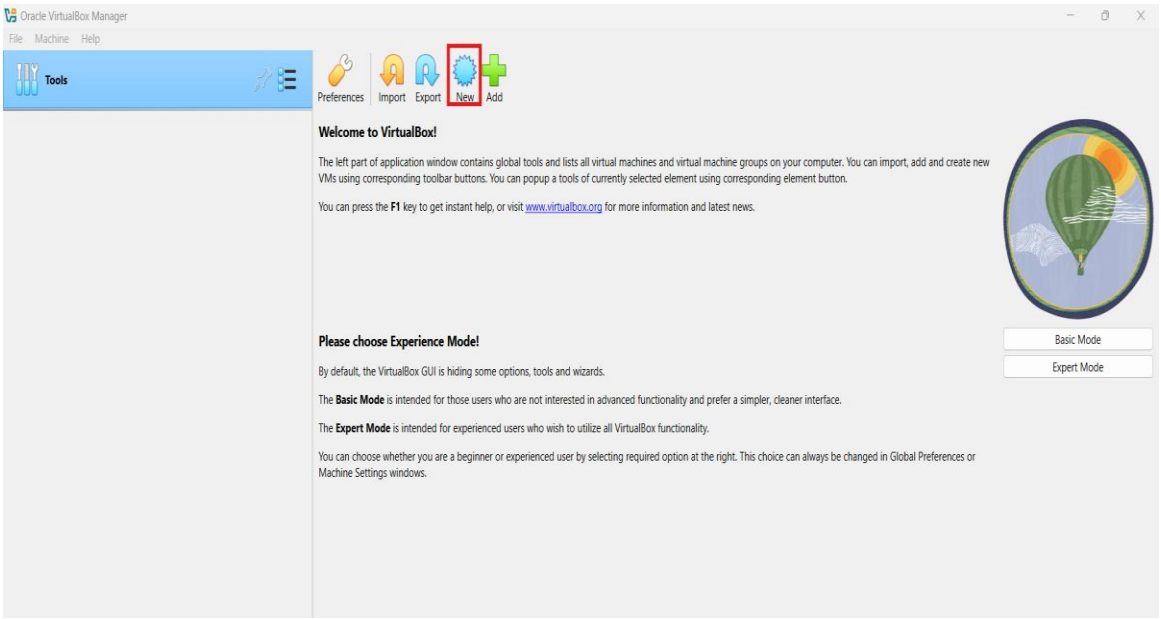


Figure 10 – Portal to download Ubuntu 25.04

2. Launch Oracle VirtualBox, click new, enter Name: Ubuntu 25.04, Type: Linux, Version: Ubuntu (64-Bit) & click next.



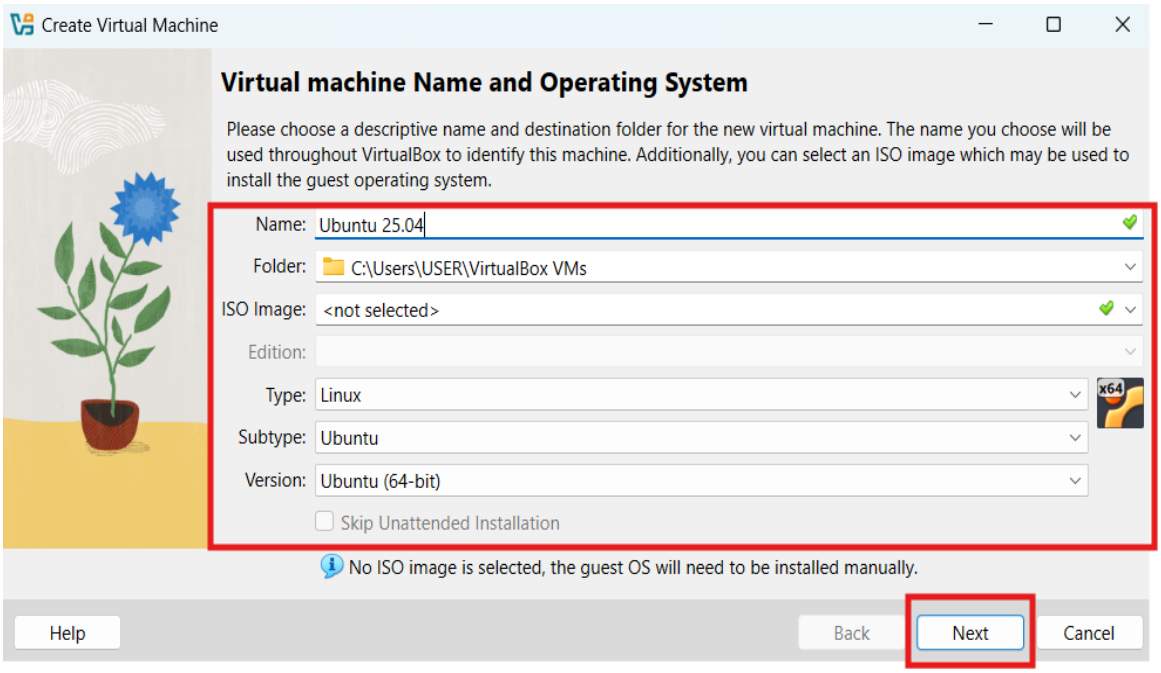


Figure 11 – Creation of new Virtual Machine

3. Assign 4GB of memory to VM & click next.

Base Memory: 4 GB (4096 MB)
Processors: 1 CPU

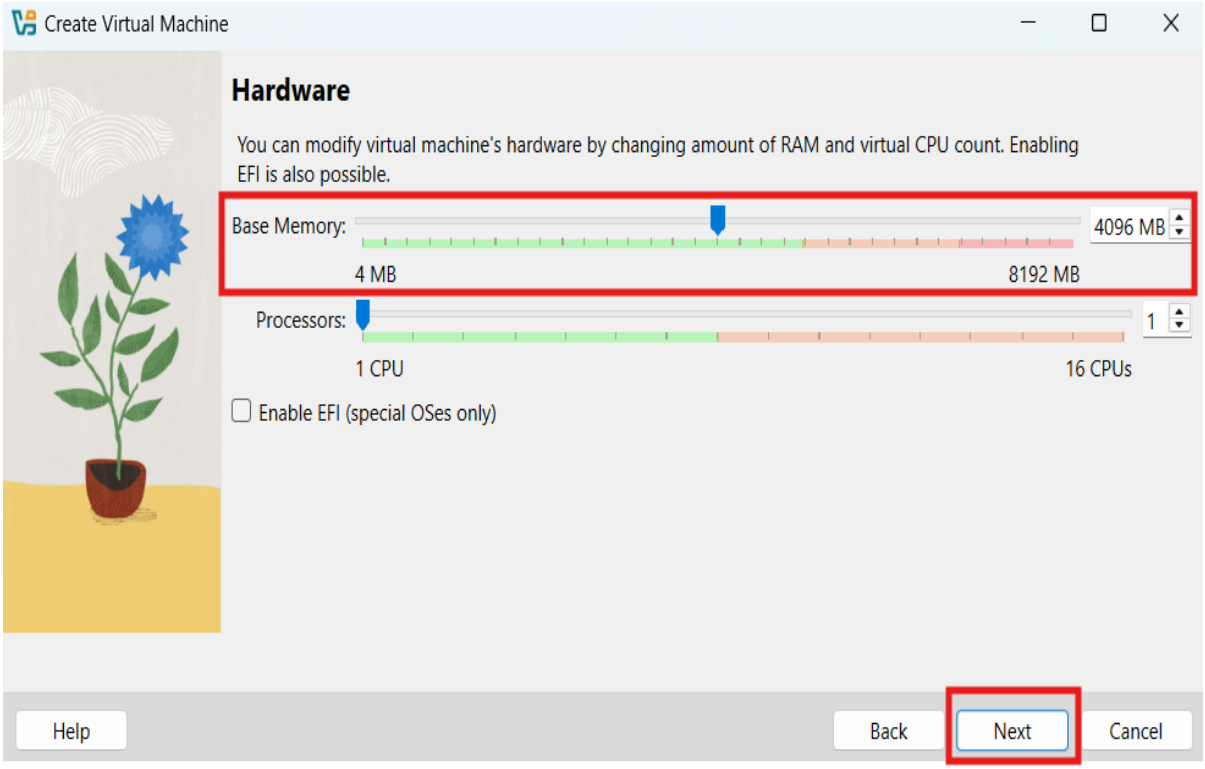


Figure 12 – Allocation of RAM & CPU to Virtual Machine

4. Assign 25GB of space to Virtual Hard Disk & click next.

Hard Disk Size: 25 GB

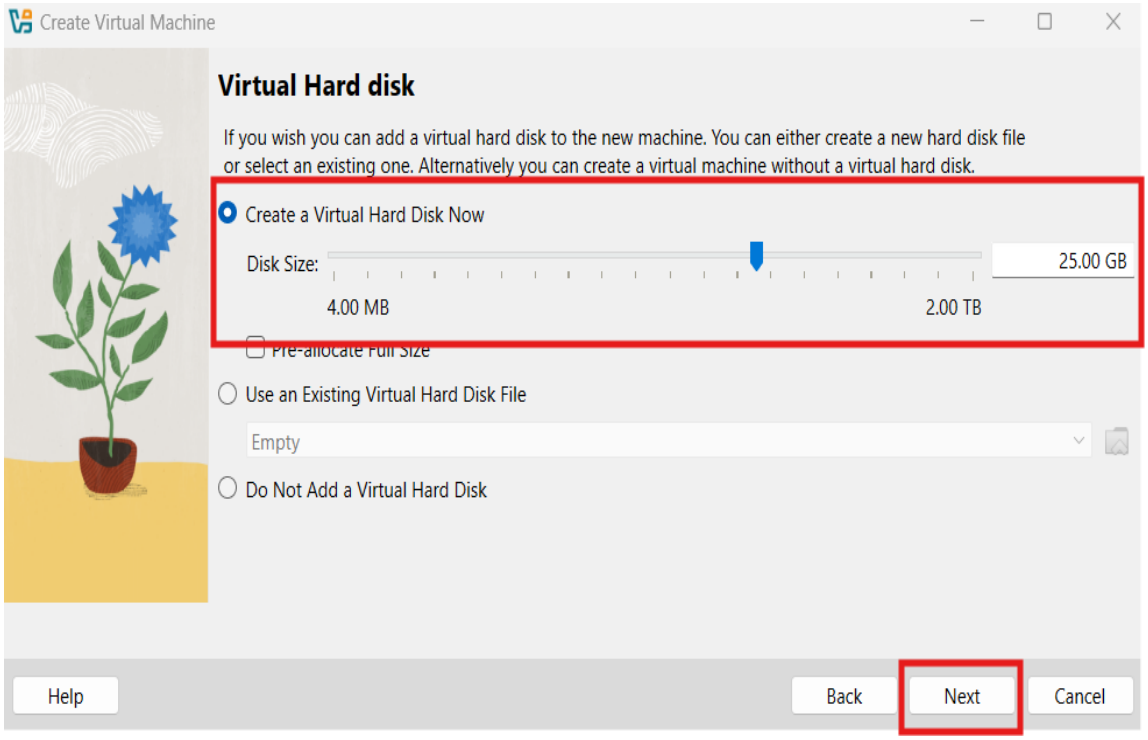


Figure 13 – Allocation of Hard Disk Memory to Virtual Machine

5. Review all the configurations for the new virtual machine & click finish.

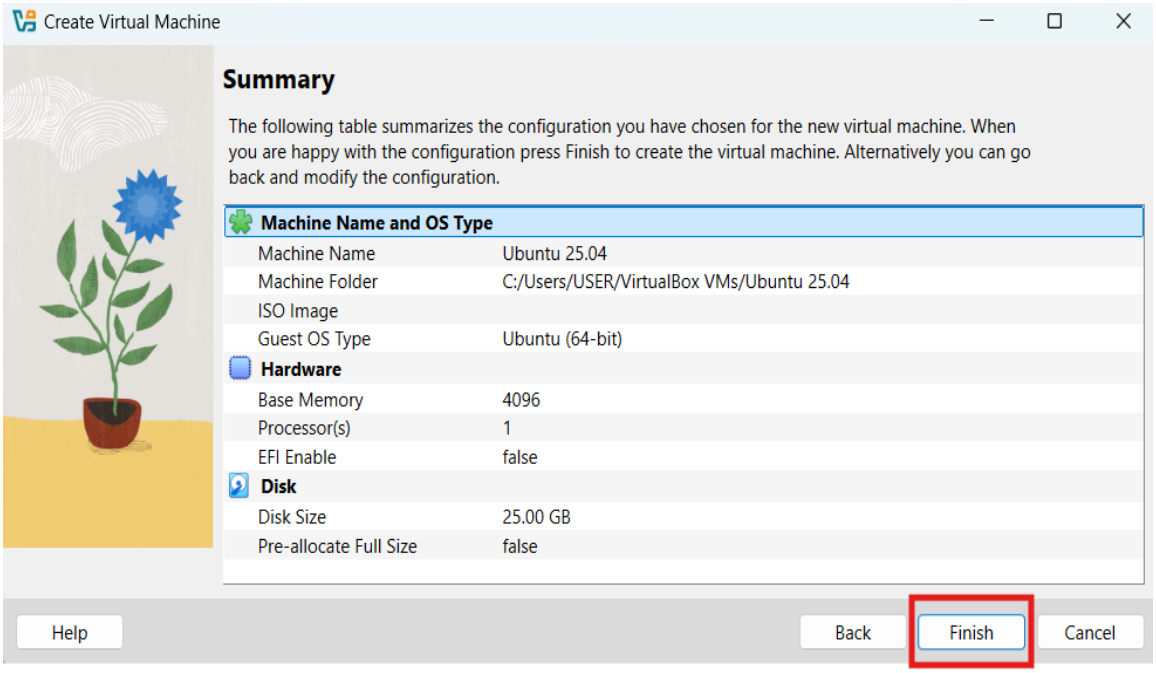


Figure 14 – Verification of defined configurations

6. **Select Ubuntu VM from the left panel, click Settings -> Storage -> Controller IDE -> Empty -> Choose a Disk File -> Browse the Ubuntu 25.04 - > Ok.**

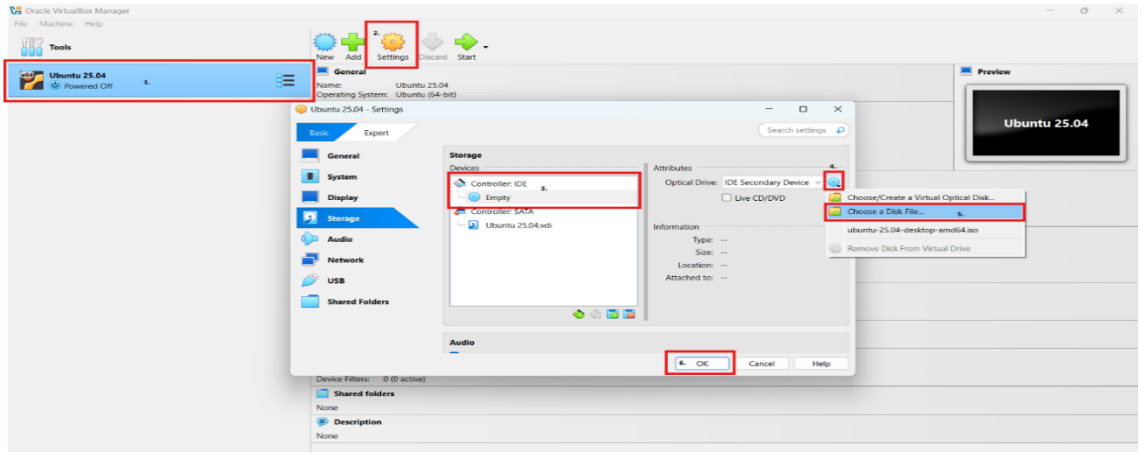


Figure 15 – Loading of Ubuntu ISO file in Virtual Machine

7. **Select Ubuntu VM, click start. After VM will boot from the ISO, select Install Ubuntu on the Ubuntu welcome screen.**

8. **Select installation configurations such as language, region, username, password & click install.**

9. **Ubuntu will install (takes 20-30 minutes depending upon the system performance). Once done, click restart now & login with the username password created at the time of installation. Now, Ubuntu in Oracle VM is ready to use.**

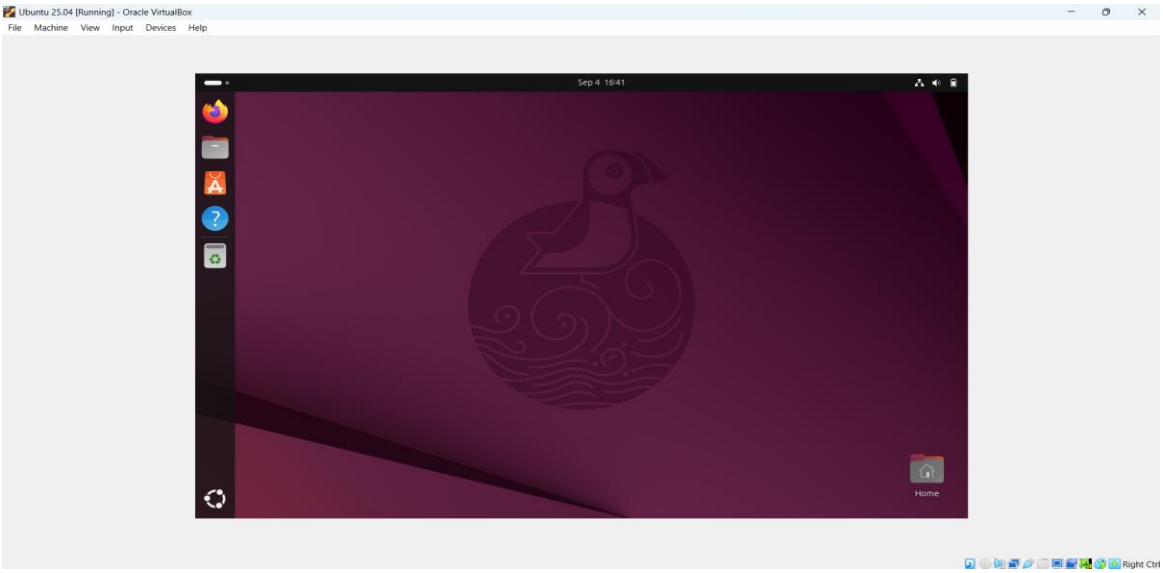


Figure 16 – Ubuntu 25.04 is ready to use on Virtual Machine

V. SETUP PROGRAMMING ENVRIONMENTS ON UBUNTU 25.04 VM USING TERMINAL

1. Install Python

sudo apt install -y python3 python3-pip

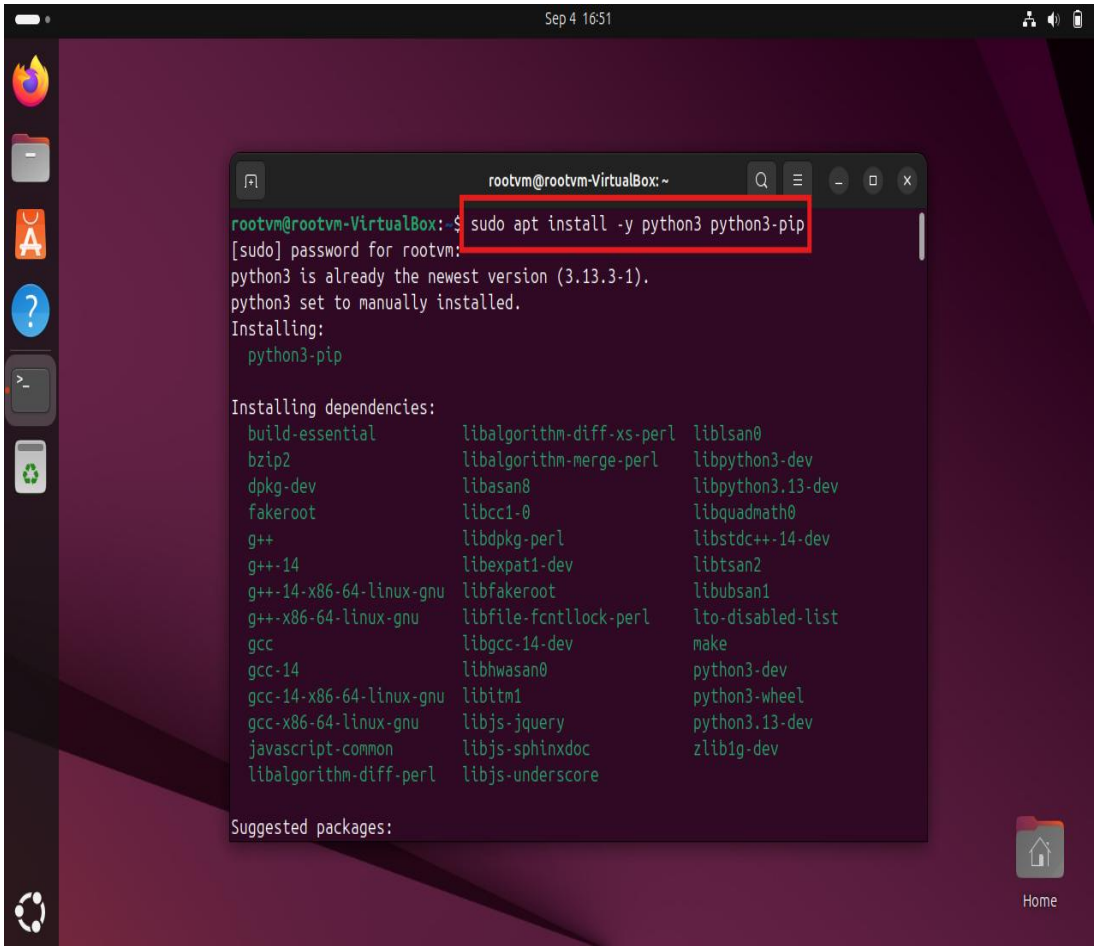


Figure 17 – Installation of Python in Ubuntu Linux

2. Install Julia

To install Julia in Ubuntu linux, curl is required so below sequence of commands need to be executed:
sudo apt install curl
curl -fsSL https://install.julialang.org | sh
sudo snap install julia --classic

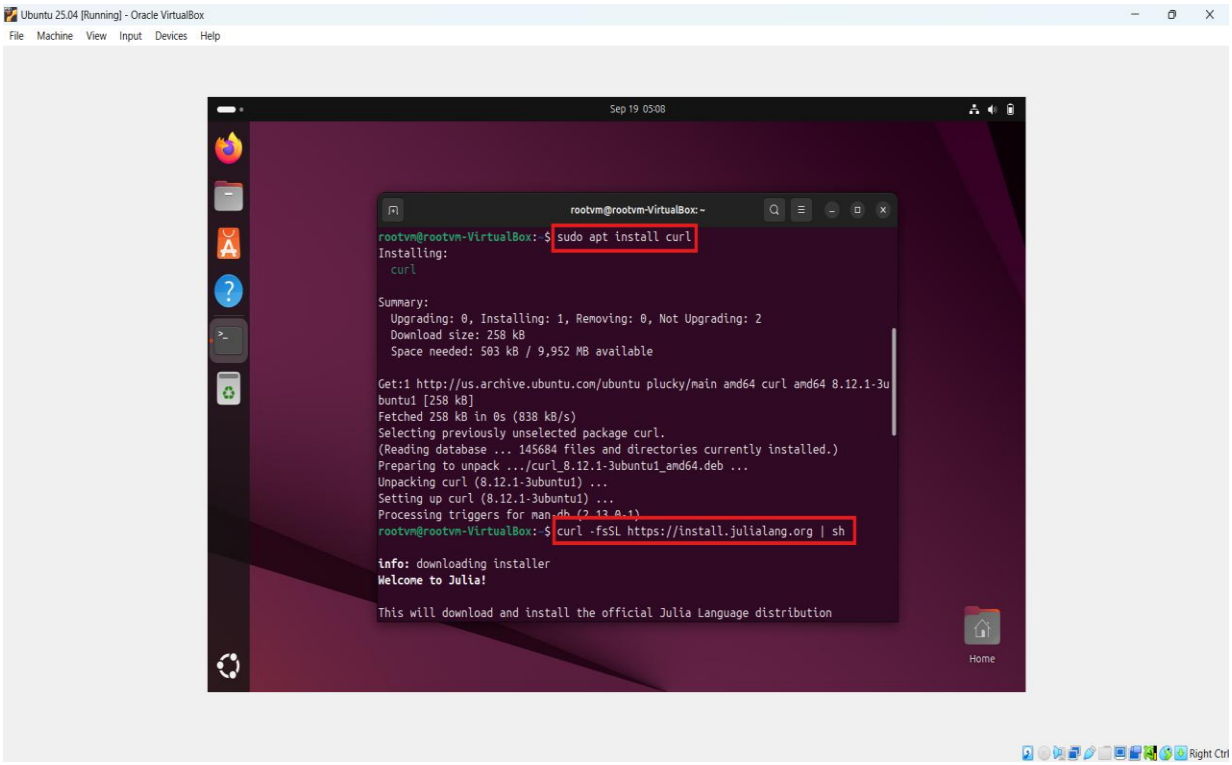


Figure 18 – Installation of Curl in Ubuntu Linux

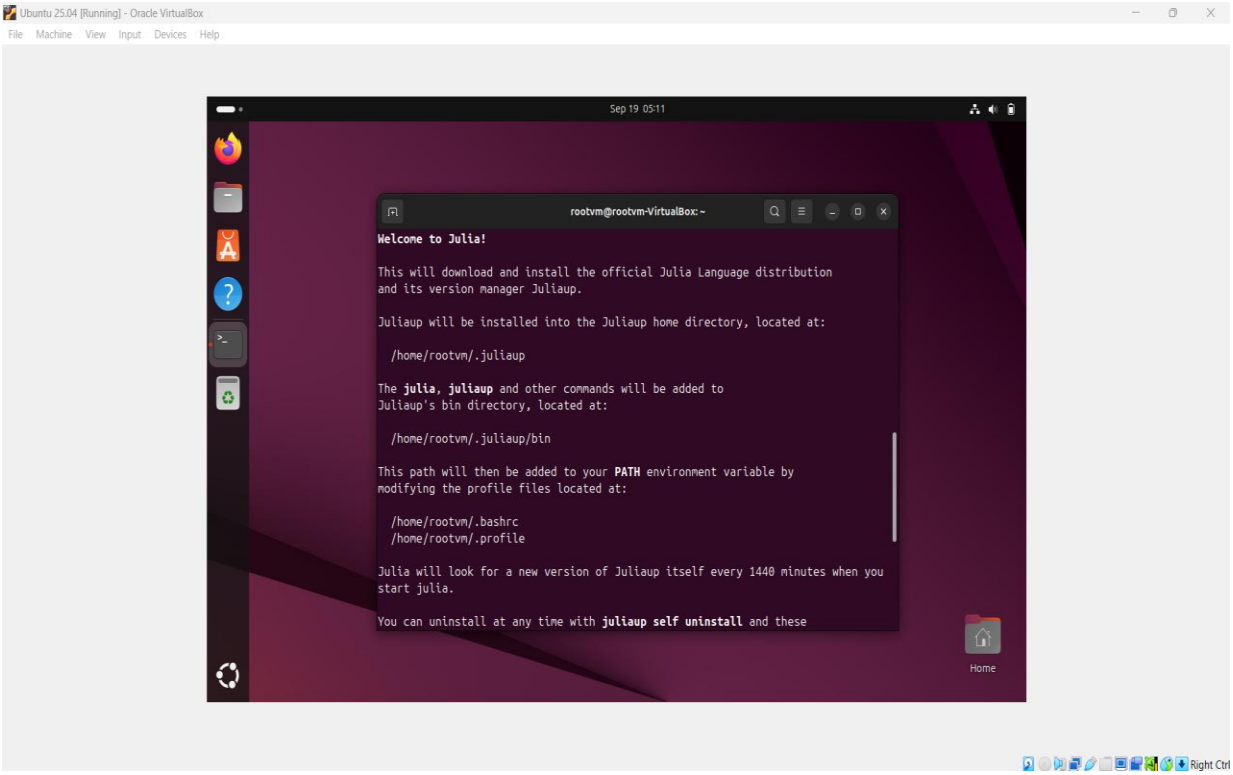


Figure 19 – Extraction of Julia in Ubuntu Linux

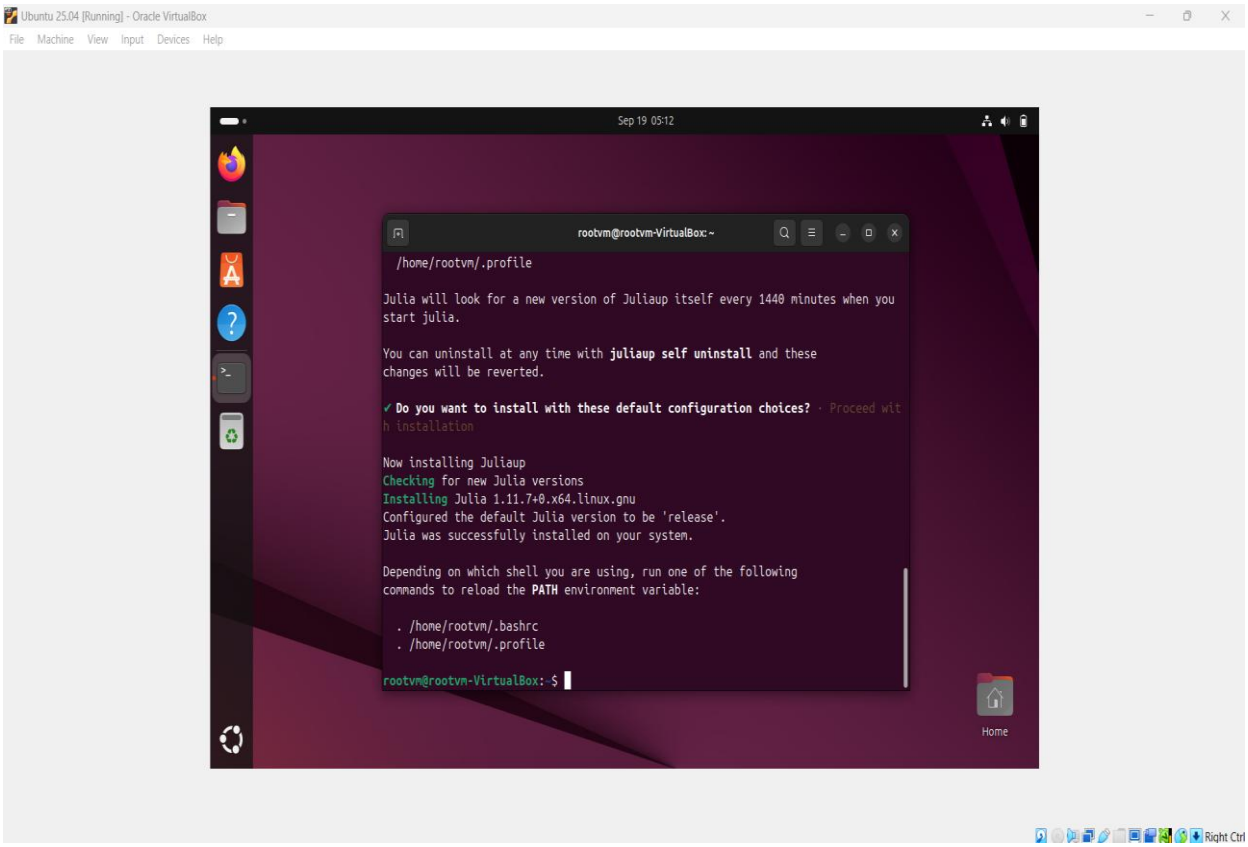


Figure 20 – Processing of Julia file in Ubuntu Linux

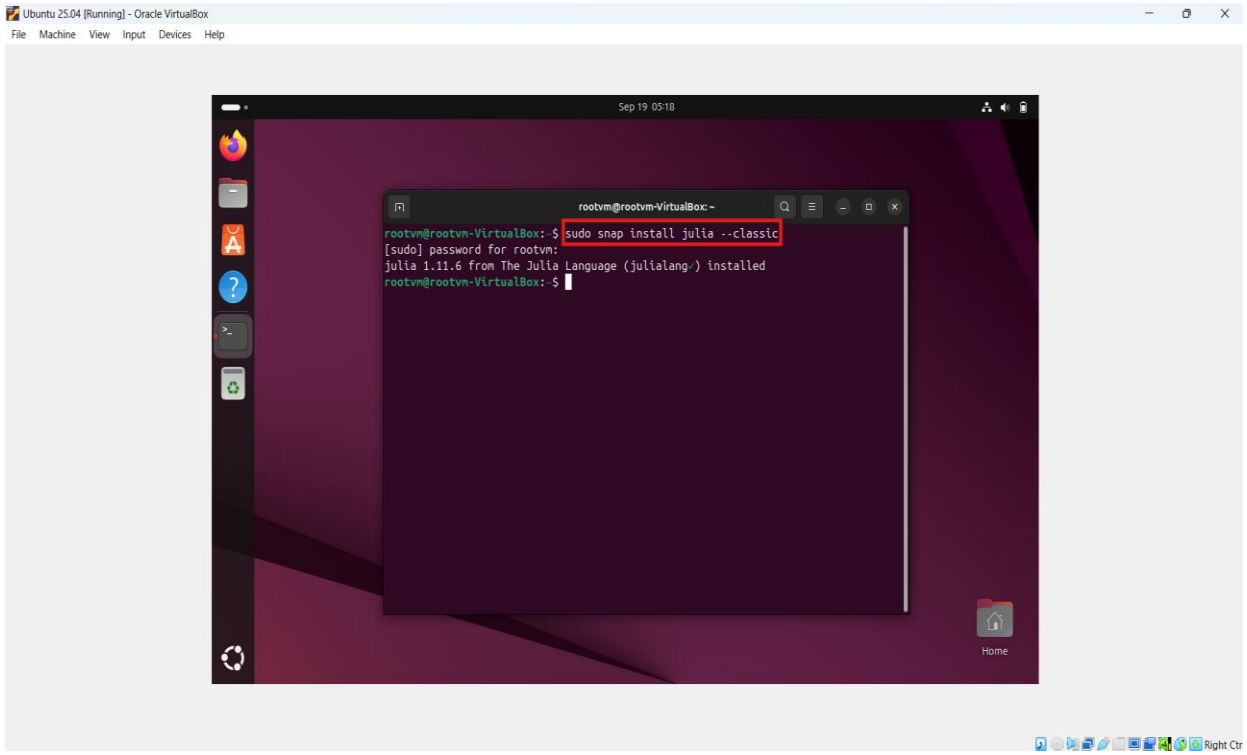


Figure 21 – Installation of Julia in Ubuntu Linux

3. Install VS Code

sudo snap install code –classic

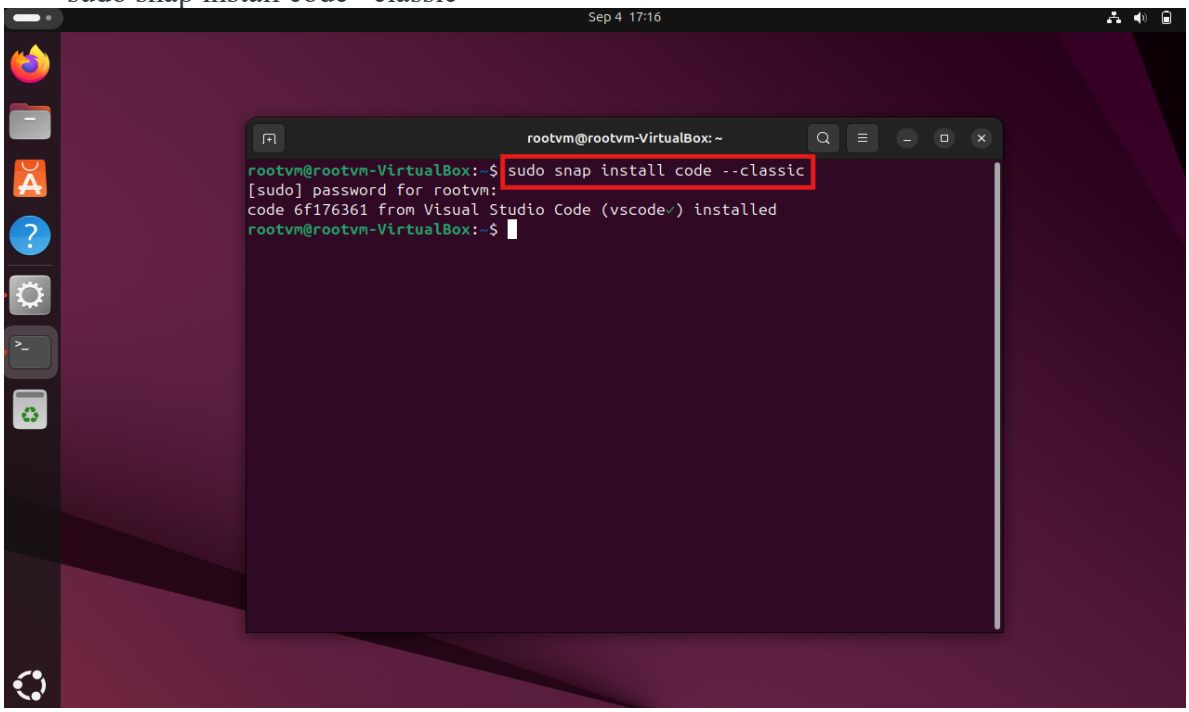


Figure 22 – Installation of Visual Studio Code in Ubuntu Linux

VI. CONFIGURATIONS VALIDATION

1. Python

Command to check whether Python is installed or not: python3

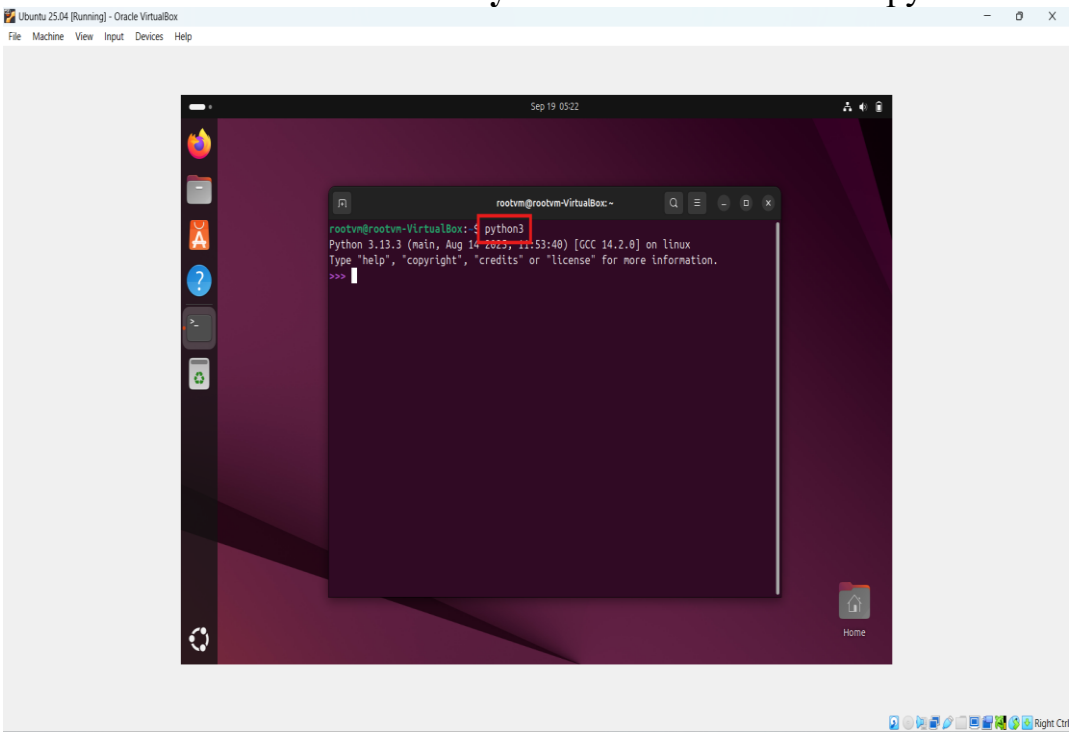


Figure 23 – Verification of Python in Ubuntu Linux

Command to check installed version of Python: python3 --version

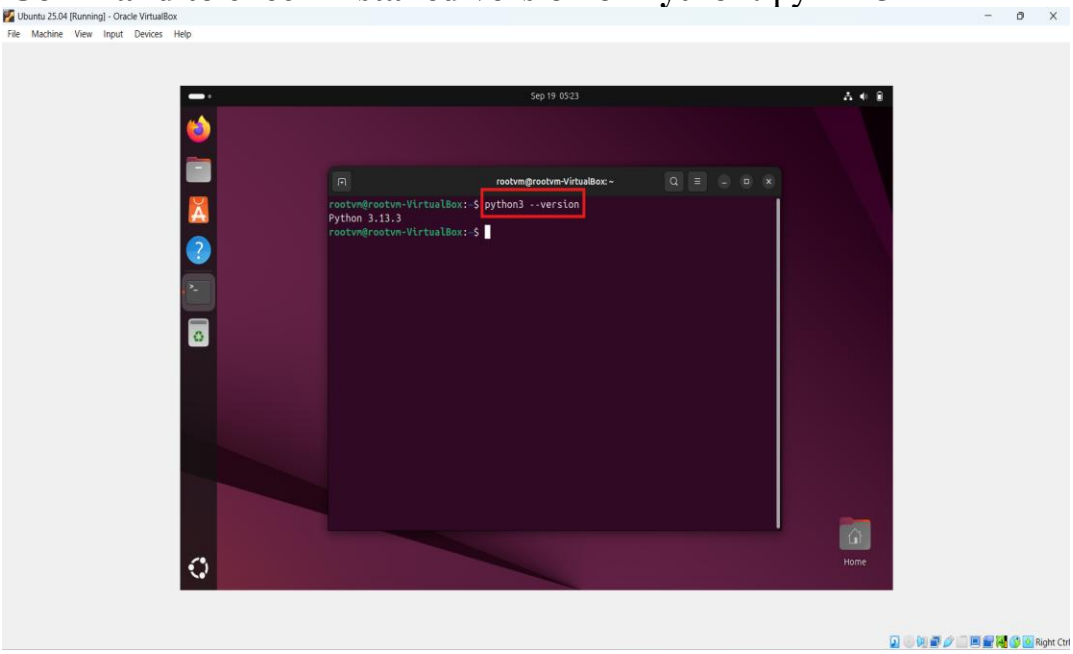


Figure 24 – Verification of Python version in Ubuntu Linux

2. Julia

Command to check whether Julia is installed or not: julia

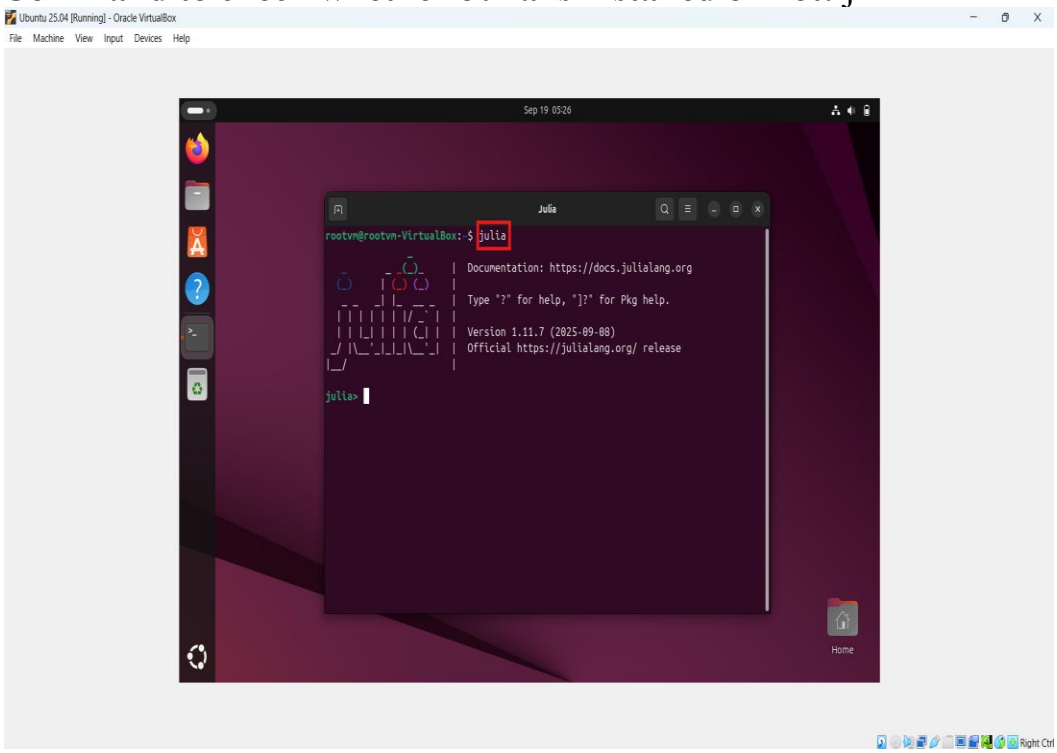


Figure 25 – Verification of Julia in Ubuntu Linux

Command to check installed version of Julia: julia -version

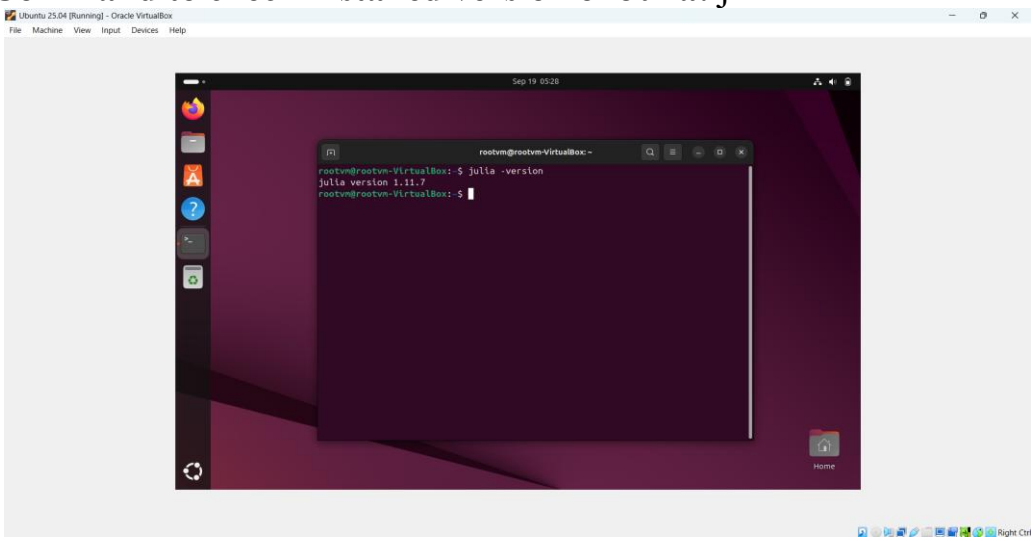


Figure 26 – Verification of Julia Version in Ubuntu Linux

VII. INSTALLATION OF REQUIRED LIBRARIES

1. Install libraries in Julia

I. CSV

julia
] (] enter the package manager prompt)
add csv

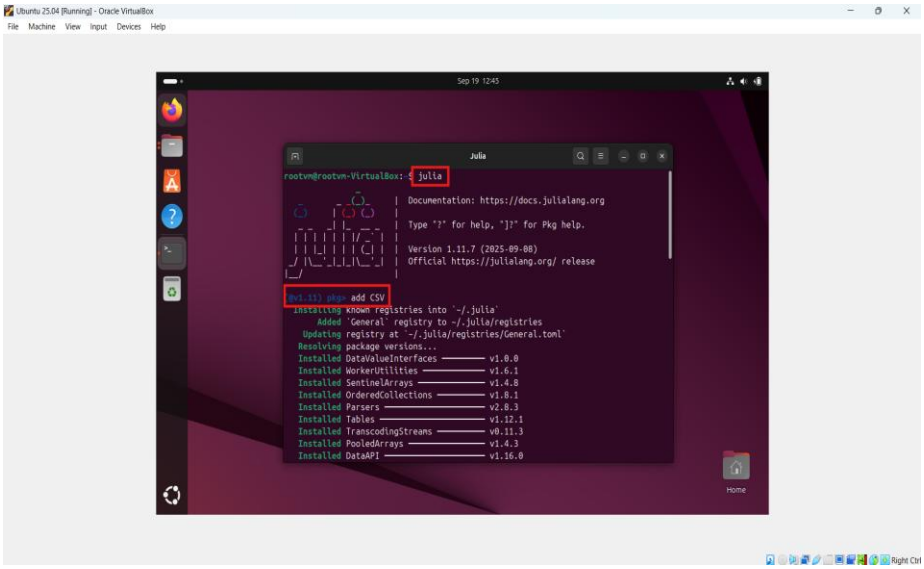


Figure 27 – Installation of Julia CSV library

II. DataFrames

julia
] (] enter the package manager prompt)
add csv

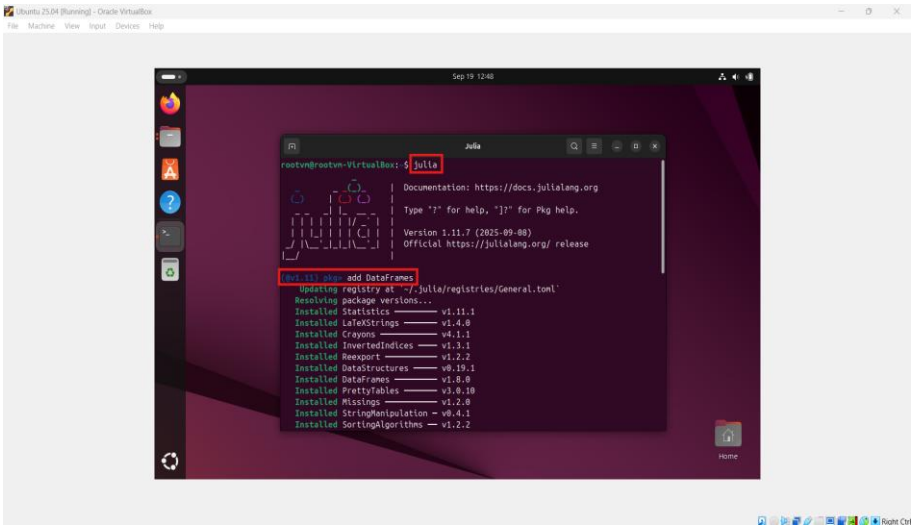


Figure 28 – Installation of Julia DataFrame library

III. Dates

julia
] (] enter the package manager prompt)
add Dates

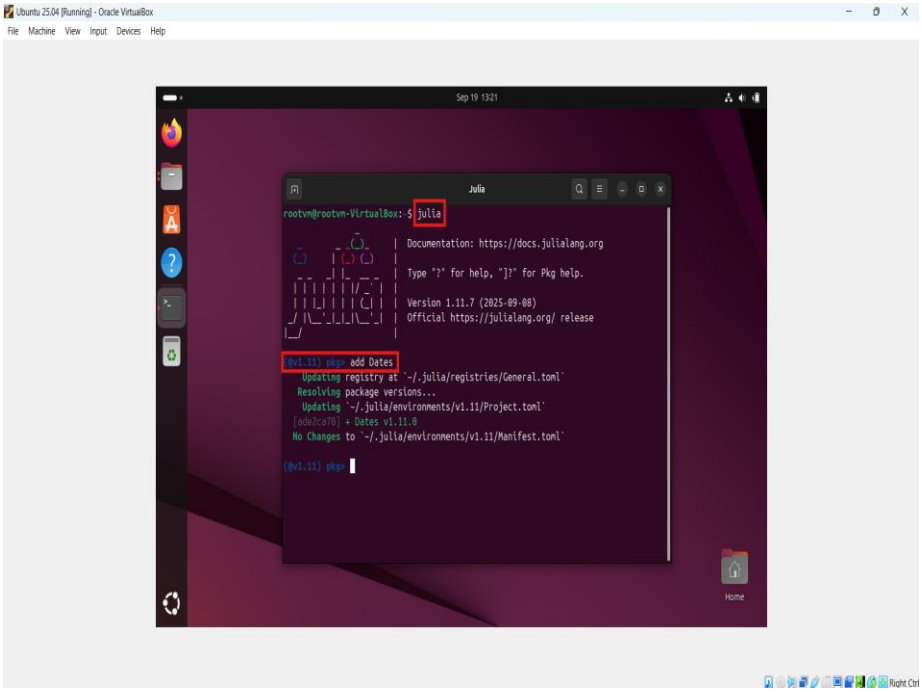


Figure 29 – Installation of Julia Dates library

2. Install libraries in Python

csv, os, and time are the built-in libraries of python. So, it does not need to install manually.

VIII. LINKED LIST DATA STRUCTURE

Linked list is a linear data structure that stores data in dynamic/non-continuous form, allows efficient insertion and deletion operations. Inside linked list, each node stores the data and memory address of the next node.

Basically, there are three types of linked list:

1. Singly Linked List

A singly linked list consists of the nodes, pointer (data, next) where each node represents the data field and points to the memory address of next node. In addition to this, head stores the memory address of the first node whereas the next of the last node is null.

Head = Memory address of 1st node

Next = Memory address of next node (Pointer)

Null = Represents the next of last node

Data = Element stored in current node

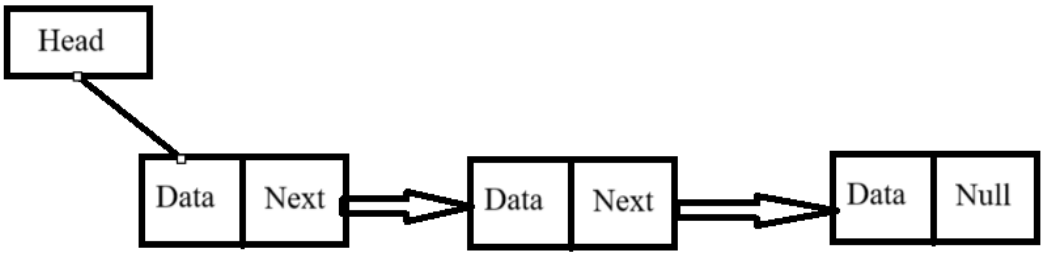


Figure 30 – Representation of Singly Linked List

2. Doubly Linked List

A doubly linked list consists of the nodes, pointers (previous, data, next) where each node represents the data field and points to the memory address of next as well as previous node. In addition to this, head stores the memory address of the first node, next of the last node is null and previous of the first node is null. A doubly linked list is a more complex data structure than a singly linked list, but it offers several advantages. The main advantage of a doubly linked list is that it allows for efficient traversal of the list in both directions.

Head = Memory address of 1st node

Next = Memory address of next node (Pointer)

Prev = Memory address of previous node (Pointer)

Null = Represents the previous of 1st node and next of last node

Data = Element stored in current node

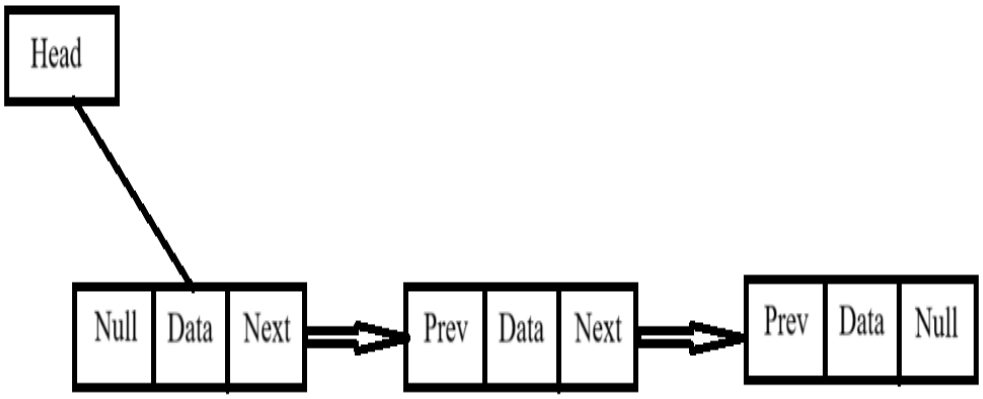


Figure 31 – Representation of Doubly Linked List

3. Circular Linked List

A circular linked list is a data structure where a last node points to the address of first node or head, forms a closed loop. It is best suitable for tasks like scheduling and managing playlists.

There are two types of circular linked list:

a. Circular Singly Linked List

A circular singly linked list consists of the nodes, pointer (data, next) where each node represents the data field and points to the memory address of next node. In addition to this, head stores the memory address of the first node whereas the next of the last node stores the memory address of the first node, forms a circle.

Head = Memory address of 1st node

Next = Memory address of next node (Pointer)

Data = Element stored in current node

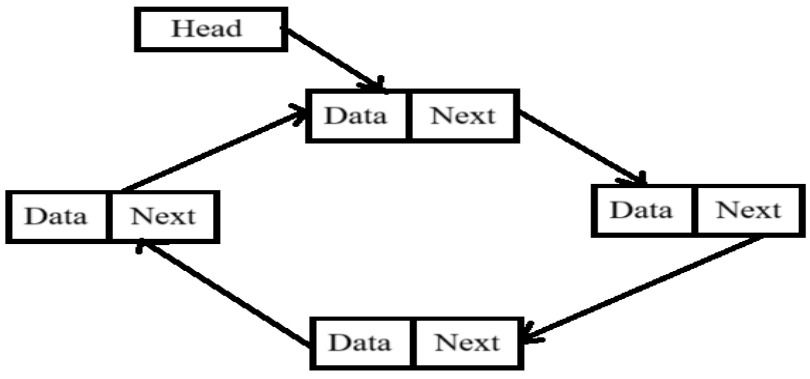


Figure 32 – Representation of Circular Singly Linked List

b. Circular Doubly Linked List

A circular doubly linked list consists of the nodes, pointers (previous, data, next) where each node represents the data field and points to the memory address of next node. In addition to this, head stores the memory address of the first node, next of the last node stores the memory address of the first node, and previous of the first node stores the memory address of the last node, forms a circle.

Head = Memory address of 1st node

Next = Memory address of next node (Pointer)

Prev = Memory address of previous node (Pointer)

Data = Element stored in current node

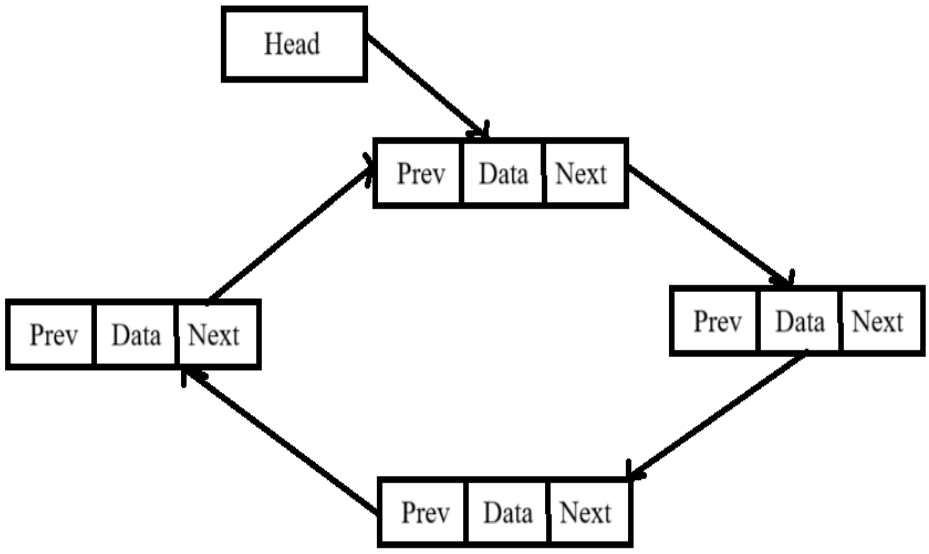


Figure 33 – Representation of Circular Doubly Linked List

IX. RECURSION

A technique with the help of which function calls itself either directly or indirectly is called recursion and its relevant function is called recursion function. A recursion algorithm continues to call itself until all the sub-problems gets resolved to provide the combined solution.

Process to Execute Recursion Approach

1. Define a base class to stop the condition defined in recursive function in order to prevent from infinitely calling itself.
2. Break a problem into the smaller versions and then define those sub-problems into the recursive functions to solve each.
3. Merge the solutions of smaller problems to achieve solution for an original problem.

X. BUBBLE SORT

Bubble sort is a comparison-based sorting algorithm where two adjacent elements are compared and swapped if they are not in sequence in order to arrange all the elements in ascending or descending order as per the requirement.

1. Complexity

- **Best Case** – O(n) -> When an array is already sorted.
- **Average Case** – $O(n^2)$ -> In case of lots of comparison and swaps.
- **Worst Case** – $O(n^2)$ -> When an array is in reverse order.
- **Space Complexity** – O(1) -> No extra memory required.

2. Algorithm

Step 1 – Check if the first element of input is greater than the second element.

Step 2 – If greater, swap the elements and move the pointer forward.

Step 3 – Repeat step 2 until the last element.

Step 4 – Check of all the elements are sorted. If not, repeat the same process (step 1 to step 3) from the last element to first and output is the sorted array.

3. Pseudo Code

```
Bubble_Sort(array)
for i -> 1 to length[array] do
   for j -> length [array] down-to i +1 do
      if A[j] < A[j-1] then
         Swap A[j] <-> A[j-1]
```

4. Example

5	**10**	**7**	**21**	**1**
5	**10**	**7**	**21**	**1**
5	**7**	**10**	**21**	**1**
5	**7**	**10**	**21**	**1**
5	**7**	**10**	**1**	**21**
5	**7**	**10**	**1**	**21**
5	**7**	**10**	**1**	**21**
5	**7**	**1**	**10**	**21**
5	**7**	**1**	**10**	**21**
5	**7**	**1**	**10**	**21**
5	**1**	**7**	**10**	**21**
1	**5**	**7**	**10**	**21**

XI. INSERTION SORT

Insertion sort is a simplest method of sorting elements in ascending or descending order where a sub-array is managed step by step by taking one item at a time and placing it in its correct.

1. Complexity

- **Best Case** – O(n) -> When an array is already sorted.
- **Average Case** – $O(n^2)$ -> In case of lots of swaps.
- **Worst Case** – $O(n^2)$ -> When an array is in reverse order.
- **Space Complexity** – O(1) -> No extra memory required.

2. Algorithm

Step 1 – If the first element is already sorted, return 1 and move to next element.

Step 2 – Compare with all the elements of sub-array.

Step 3 – Shift all the elements in the sorted sub-array that is greater than the value to be sorted.

Step 4 – Insert the value and repeat the steps until array is sorted.

3. Pseudo Code

```
Insertion_Sort(array)
for i -> 2 to length[array] do
   key = array[i]
   j = i -1
  while j > 0 and array[j] > key do
```

array[j+1] = array[j]

j = j – 1

array[j+1] = key

4. Example

5	**10**	**7**	**21**	**1**
5	**10**	**7**	**21**	**1**
5	**7**	**10**	**21**	**1**
5	**7**	**10**	**21**	**1**
5	**7**	**10**	**21**	**1**
5	**7**	**10**	**1**	**21**
5	**7**	**1**	**10**	**21**
5	**1**	**7**	**10**	**21**
1	**5**	**7**	**10**	**21**

XII. CPU/DISK USAGES

CPU usages is basically the percentage of the processing power allocated to a task where as disk utilization illustrates the percentage of the hard drive or SSD is occupied by that task to perform read or write operations.

In windows, Task Manager is the best feature to monitor the real time memory, CPU and disk utilization. Moreover, this monitoring can also help in evaluating the system performance. In organizations, several teams are built to monitor these utilization parameters just to make sure that application will work smoothly.

There are several Linux commands that are executed in terminal to calculate CPU/DISK Utilization:

- **top**: system performance, including CPU and memory utilization.
- **mpstat**: CPU usage statistics.
- **iostat**: monitor systems input output device statistics including disk performance.
- **df**: display information about disk space usage.

XIII. CLASSES/FUNCTIONS DESCRIPTION

1. Node Class:

It defines the parameters of the linked list like data, next, and previous.

2. singly_linked_list/doubly_linked_list Class:

Here, we are using the concept of singly linked list in Julia programming and doubly_linked_list in Python programming. It defines the parameter of first node, last node and size of the linked list. In addition to this, several recursive functions are also defined in this class.

i) add_node():

This function is used to insert a new node in the linked list to store dynamic records.

ii) remove_node():

This function is used to delete a new node from the linked list.

iii) find_by_id():

It is used to find the

iv) display_list():

This function displays all the present number of nodes inside linked list.

3. memory_database Class:

This class implements all the recursive function that corresponds to the memory database.

i) load_csv():

With the help of this function, all the records of the student-data.csv are switched to the nodes of linked list which means each row inside CSV file is considered as each node. If there are n number of rows in file then n number of nodes get created.

ii) load_data():

This function is responsible for loading all the nodes/records to the memory database but 1st row is considered as header of memory database because first node of linked list holds the column names of CSN file & actual data starts from 2nd node.

iii) export_csv():

This function basically extracts the data from memory database.

iv) export_to_csv():

Using this function, all the extracted data from memory database is exported to the resultant csv file

v) display_data():

This function prints all the records available in memory database.

vi) monitor_changes():

This function monitors the real time alterations in the student-data.csv to synchronize memory database with the input file. For instance, if any of the record is inserted or deleted from the input file then this function identify that change & update those alterations inside memory database. This function is disabled in the given code. To execute this function, we need to uncomment this function call from the source code.

vii) add_row():

A new record can be appended at the last of memory database using this function. student-data.csv is independent from this function.

viii) remove_row():

An existing record can be removed through its row number inside memory database using this function. student-data.csv is independent from this function.

ix) validate_key():

Checks if a given column exists in a memory database and print all the columns of invalid column name is given.

x) bubble_sort():

Implements bubble sort on the linked list data by converting to an array and compares values for the chosen column. It handles both string and numeric value to arrange.

xi) insertion_sort():

Implements insertion sort on the linked list data by converting to an array and compares values for the chosen column. It handles both string and numeric value to arrange.

xii) choose_sort_key ():

This function allows user to pick a column on which the sorting algorithm is implemented.

xiii) opt_sorting_algorithm():

This function allows user to implement either bubble sort by using option 1 or insertion sort by using option 2. By default, bubble sorting is implemented if invalid input is given.

xiv) opt_sorting_algorithm():

It executed the exported sorting algorithm and export the sorted data into a csv file.

xv) get_system_statistics():

It fetched the memory as well as disk usage in the current directory.

xvi) run_shell_command():

It basically executed the Linux terminal commands.

xvii) Measure_sort_performance():

This function is responsible for calculating the difference between disk and CPU utilization.

xviii) display_performance_comparison():

It is used to compare the performance of bubble sort and insertion sort based on CPU and Disk consumption parameters.

XIV. DISCUSSION

An implementation of a toy “memory database” is bounded around three core principles: loading, adapting, and exporting. Each recursive function collectively corresponds to these core principles in order to meet the requirements.

1. Load Data Recursively

The initial step is to read student-data.csv row by row. Each row is converted into the node of linked list using recursion algorithm. This technique eliminates the requirement of loop iterations & follows the recursion to reflect the data into memory database.

2. Adapting to Updates

A mandatory feature of the memory database is to adapt the alterations that are performed in the student-data.csv. That alteration can be insertion of rows, modification of records & deletion of rows, ensuring that updates are efficiently reflected in memory. The linked list approach optimizes these operations as node can easily be inserted without reorganizing the whole dataset.

3. Exporting Data

Once all the modifications are done, all the records stored in the nodes of memory database is traversed in a resultant CSV file. This verifies that data is synchronized between the memory database and exported file.

4. Sorting Algorithm

Bubble sort and Insertion sort gradually sort the elements stored in array irrespective of the datatype of value by arranging the adjacent positions of the elements.

5. Performance Profiling and Outcome Comparison

CPU and memory usage were calculated using /usr/bin/time -v, iostat, and pidstat. Bubble sort mostly requires more CPU time due to repeated swaps across the list, whereas insertion sort completed efficient on nearly sorted or smaller datasets. Disk I/O resource is minimum in both cases since data is processed in memory; most I/O occurs during CSV read/write operations. The analysis matches with theoretical complexities: $O(n^2)$ for both sorting in worst case, but insertion sort is more faster on practical datasets.

XV. CONCLUSION

This task has successfully implemented the development of a toy “memory database” using singly linked list in Julia and doubly linked list in Python programming language. The model efficiently loads the data from student-data.csv file, adapts to the insert/update/delete operations, export the updated dataset to a resultant CSV file by preferring recursive functions, demonstrates the execution of bubble sort and insertion sort on memory database, CPU as well as disk performance to monitor the sorting efficiency and export the sorted dataset to a CSV file recursively.

This assignment highlights the practical benefit of managing dynamic data, the use of recursion in traversal and manipulation by linked list implementation in Julia and Python programming language (extended to more complex database systems), sorting data in memory database and monitoring of system health (elaborates the CPU/DISK I/O utilization).

Acknowledge

I would like to show my sincere gratitude to my Prof. Wei David Dai for his insightful suggestions and continuous support throughout this report. This work would not be possible without his guidance and motivation.

REFERENCES

1. Linked List Data Structure. (updated 2025, July 23). GeeksForGeeks. https://www.geeksforgeeks.org/dsa/linked-list-data-structure
2. Singly Linked List Tutorial. (updated 2025, September 09). GeeksForGeeks. https://www.geeksforgeeks.org/dsa/singly-linked-list-tutorial/
3. Doubly Linked List Tutorial. (updated 2025, September 10). GeeksForGeeks. https://www.geeksforgeeks.org/dsa/doubly-linked-list/
4. Introduction to Circular Linked List. (updated 2025, September 15). GeeksForGeeks. https://www.geeksforgeeks.org/dsa/circular-linked-list/
5. Introduction to Recursion. (updated 2025, August 07). GeeksForGeeks. https://www.geeksforgeeks.org/dsa/introduction-to-recursion-2/
6. Bubble Sort Algorithm. tutorialspoint, https://www.tutorialspoint.com/data_structures_algorithms/bubble_sort_algorithm.htm
7. Insertion Sort Algorithm. tutorialspoint, https://www.tutorialspoint.com/data_structures_algorithms/insertion_sort_algorithm.htm

Output

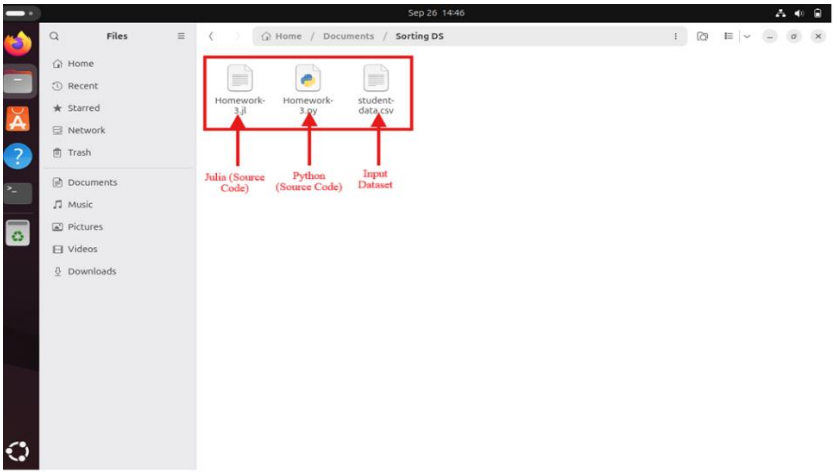


Figure 34 – Source Codes along with Dataset in Ubuntu Linux

1. Julia O/P:

a. Originally, student-data.csv file has 396 records including header. Skip header, so total records are 395.

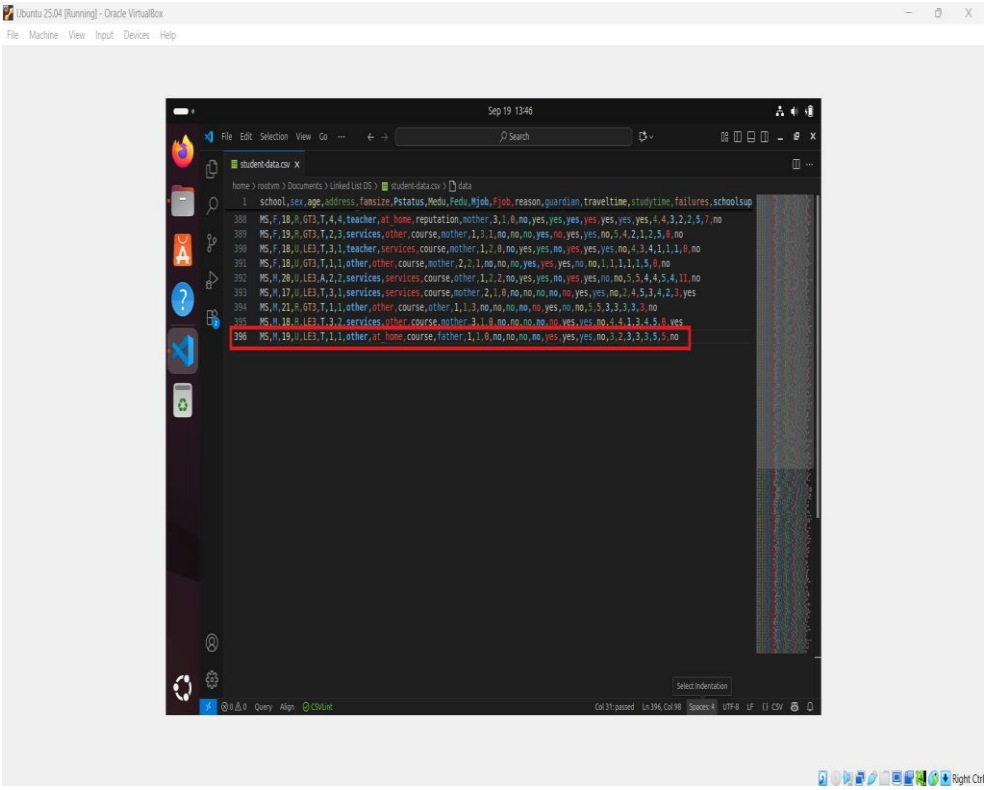


Figure 35 – Original Dataset used for Julia file

b. Function Call: load_data(), display_data(), export_to_csv("julia_result.csv"), add_row(), remove_row()
c. Execute command in terminal – julia Homework-2.jl

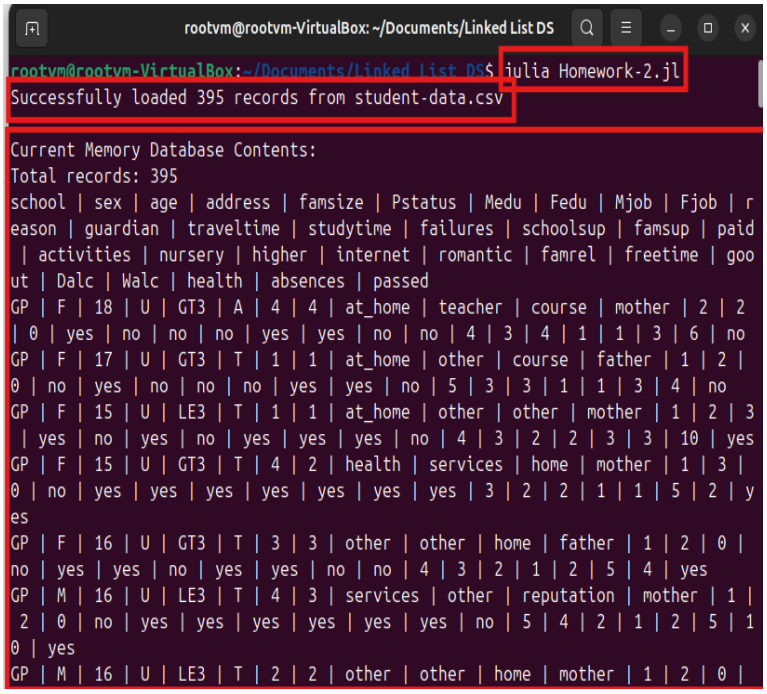


Figure 36 – Memory Database using Julia

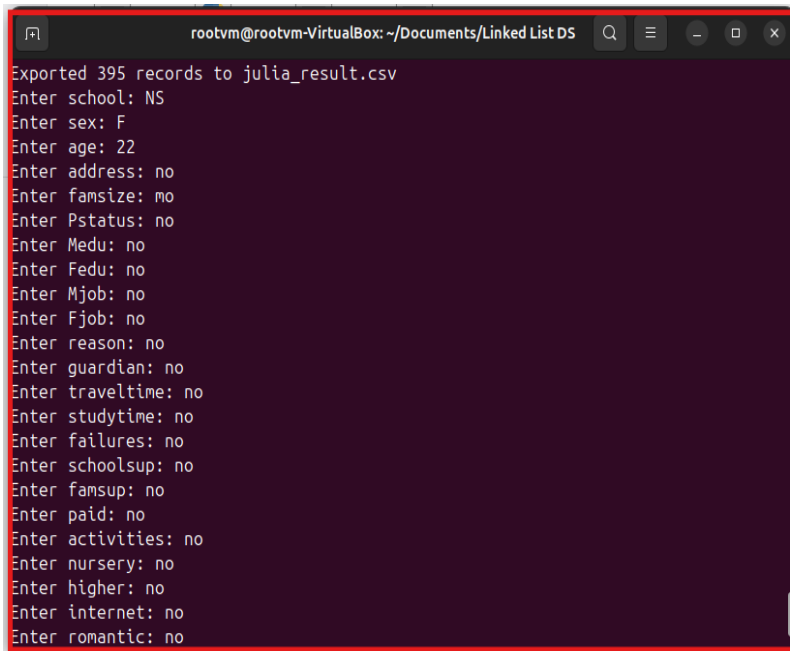


Figure 37 – Inserting data in Memory Database using Julia

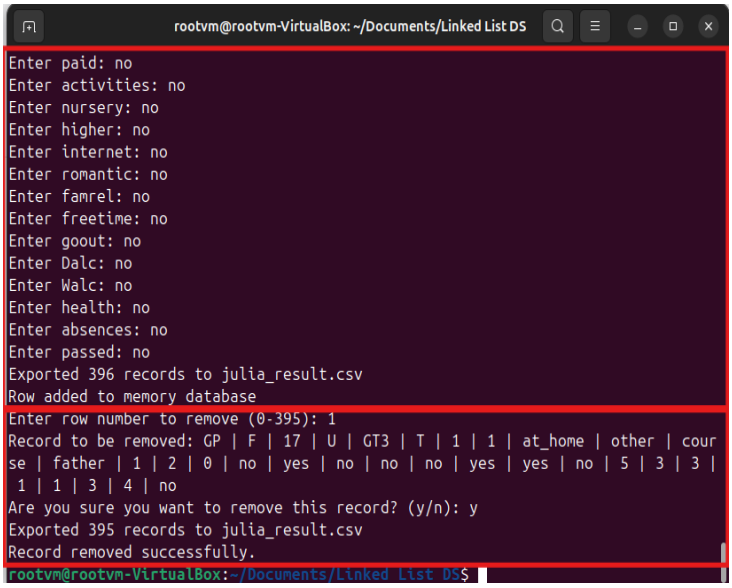


Figure 38 – Removing data from Memory Database using Julia

d. Enable monitor_changes() to get real time updations.

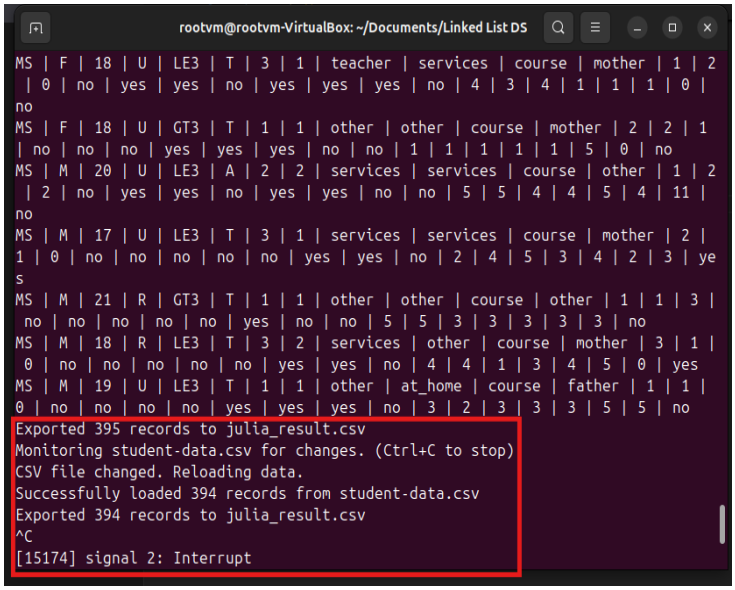


Figure 39 – Monitoring the write operation of Original Dataset using Julia

e. Implementation of Bubble Sort and Insertion Sort on the records available in memory database to sort the data using any specific key/column.

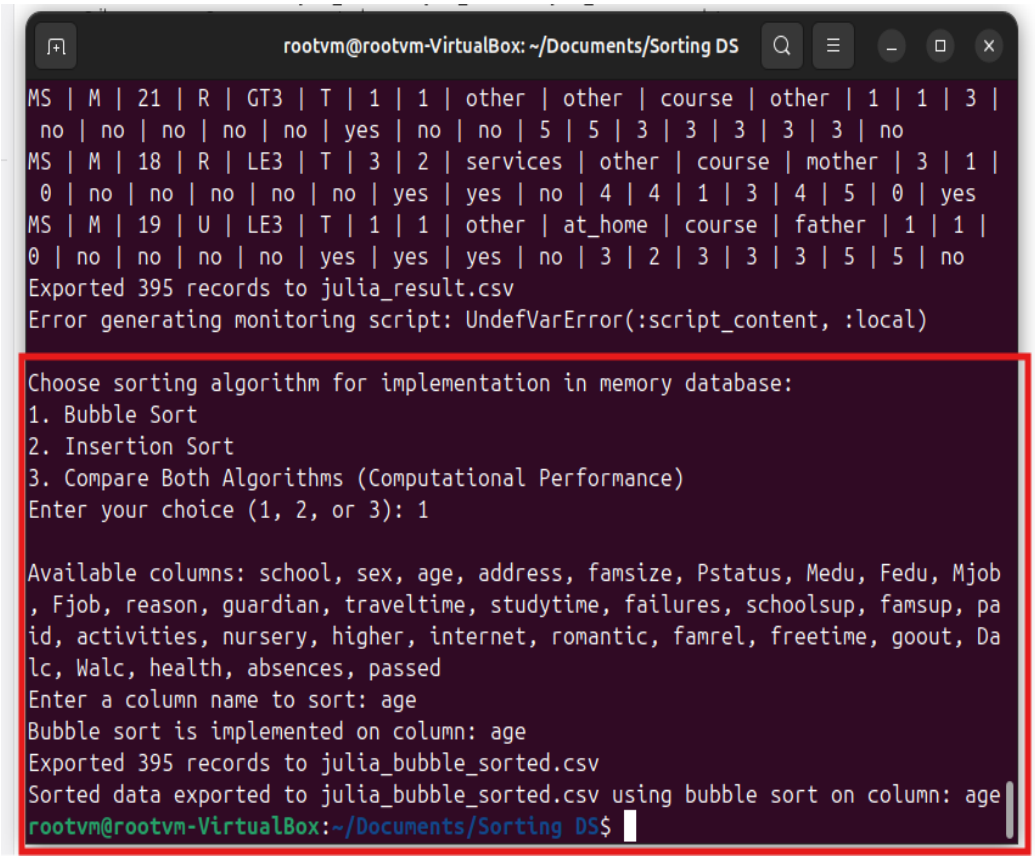


Figure 40 – Organizing records using bubble sort in Julia

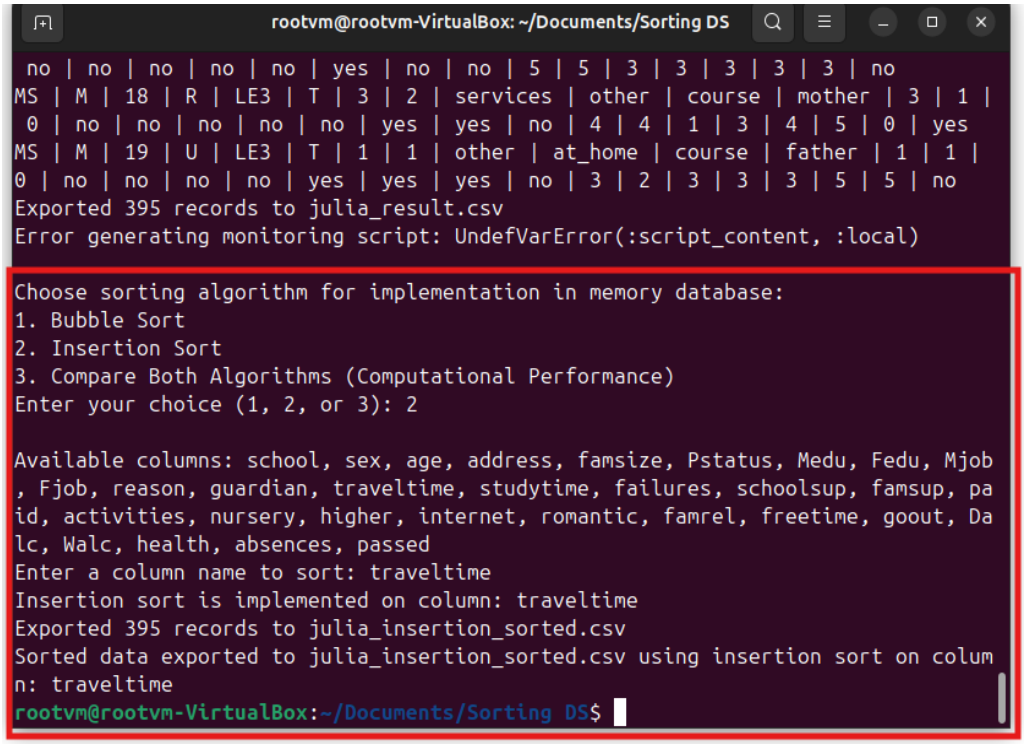


Figure 41 – Organizing records using insertion sort in Julia

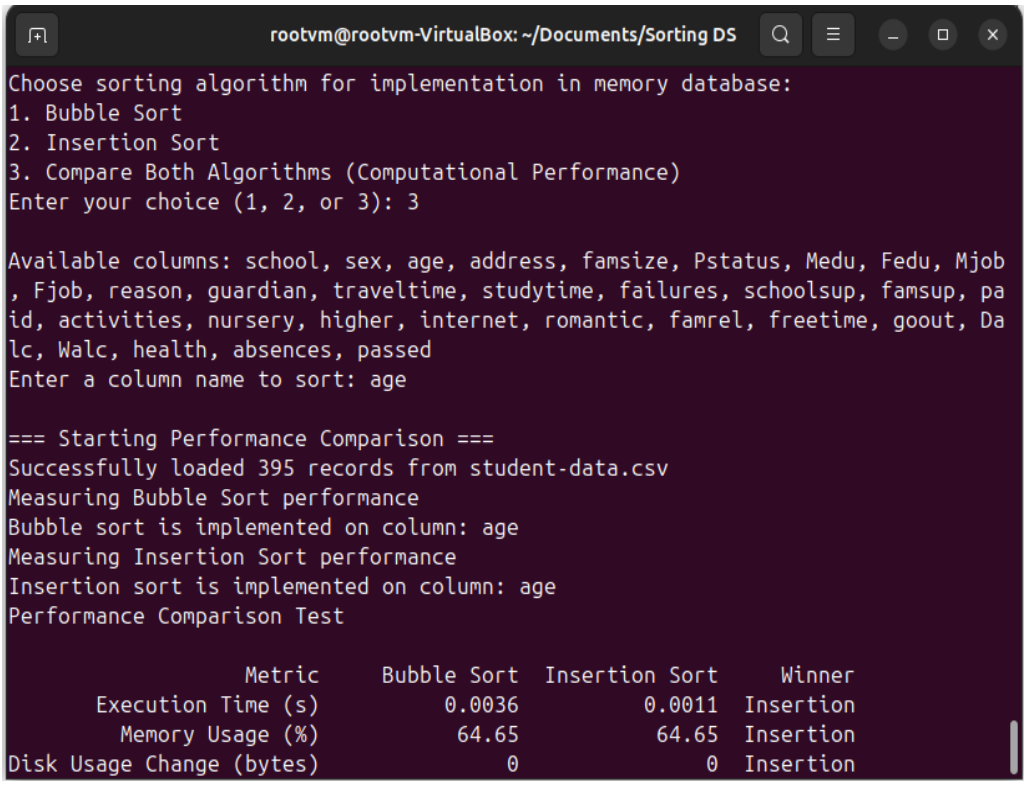


Figure 42 – Comparing CPU/Disk Consumption of both Bubble and Insertion Sort

f. Monitoring system health and comparing the CPU - Disk consumption of both sorting algorithms that can be extracted in the form of logs in .txt format.

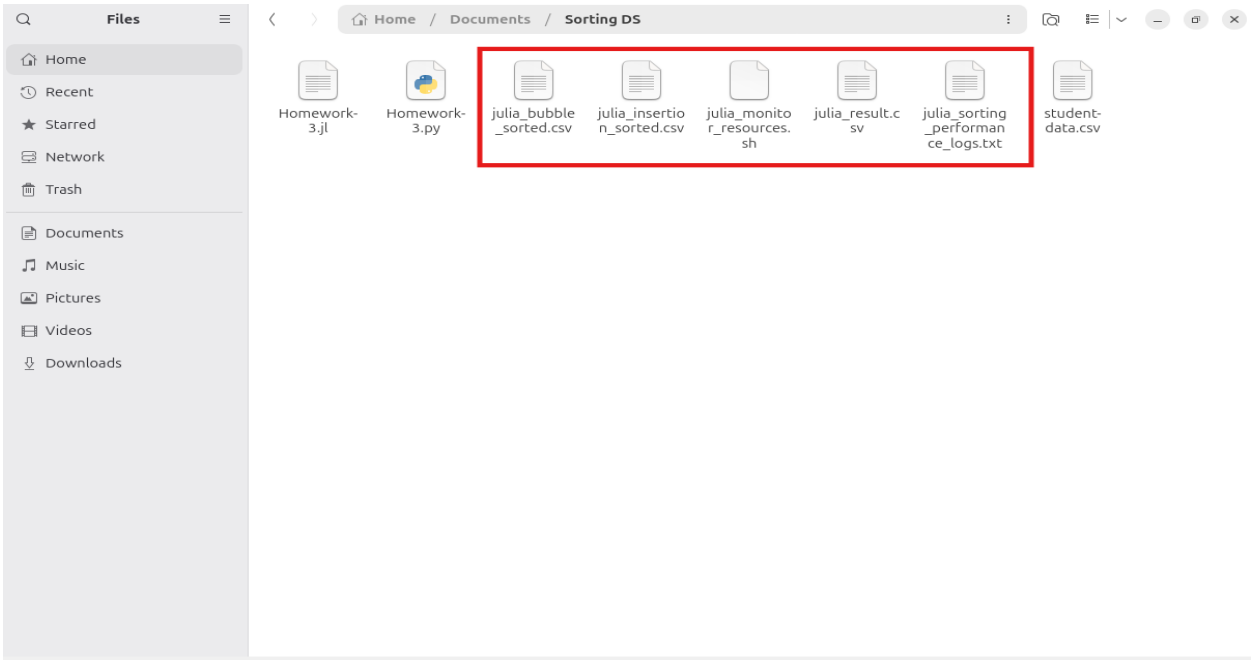


Figure 43 – Resultant files like Memory Database, Sorted and Logs using Julia

2. Python O/P:

a. Originally, student-data.csv file has 396 records including header. Skip header, so total records are 395.

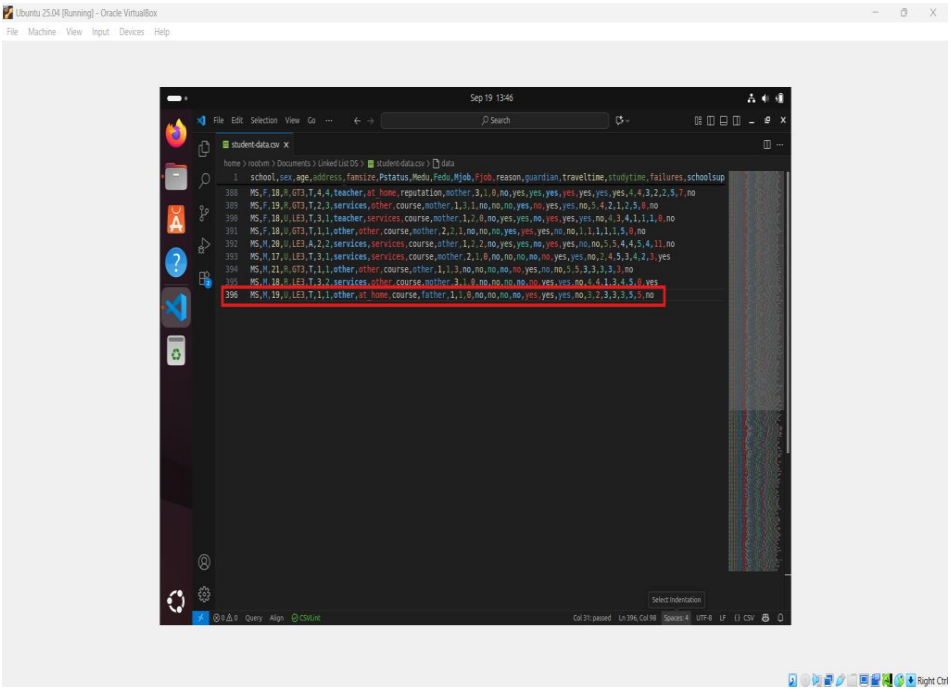


Figure 44 – Original Dataset used for Python file

b. Function Call: load_data(), display_data(), export_to_csv("python_result.csv"), add_row(), remove_row()

c. Execute command in terminal – python3 Homework-2.py

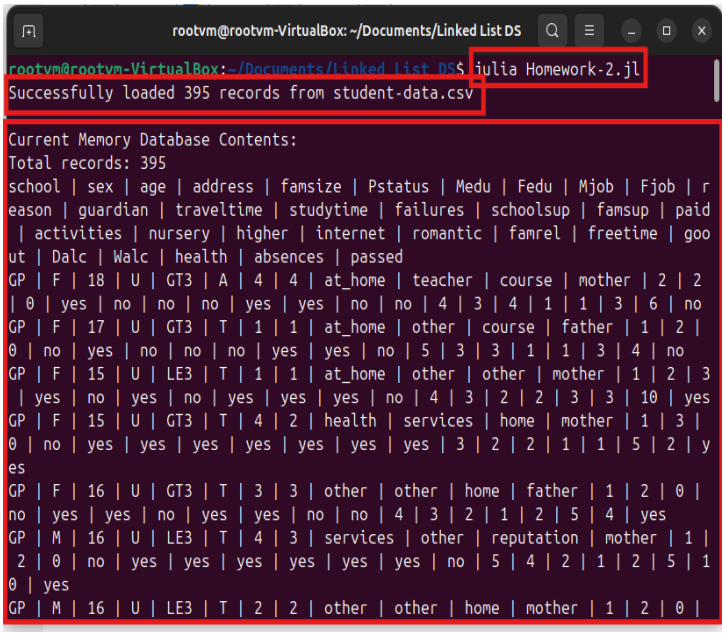


Figure 45 – Memory Database using Python

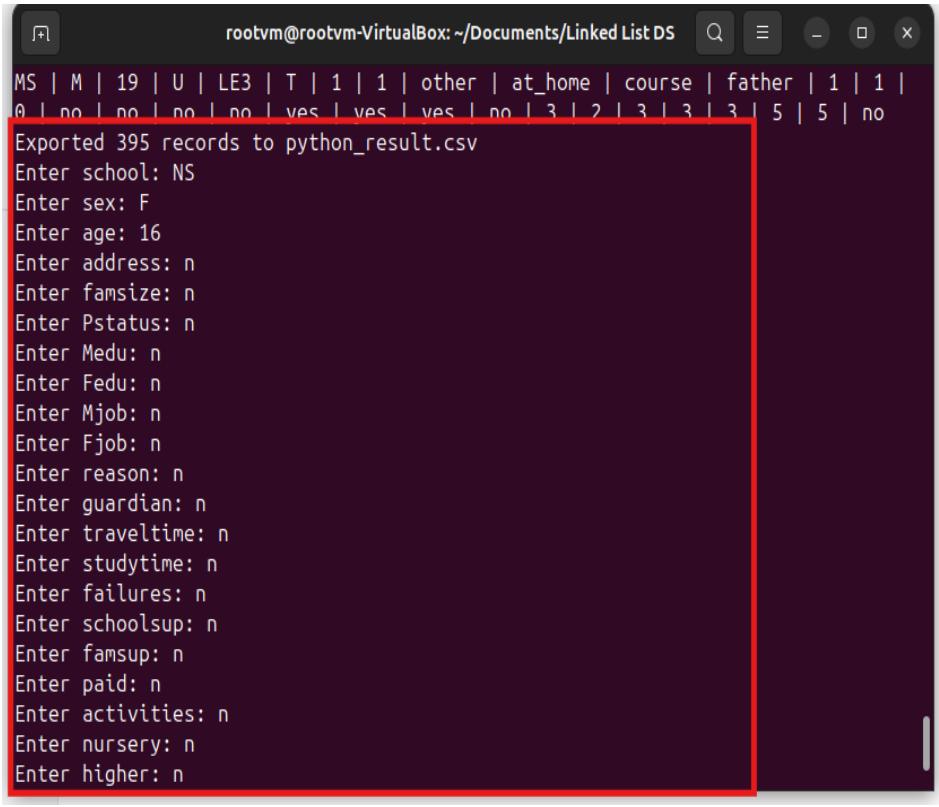


Figure 46 – Inserting data in Memory Database using Python

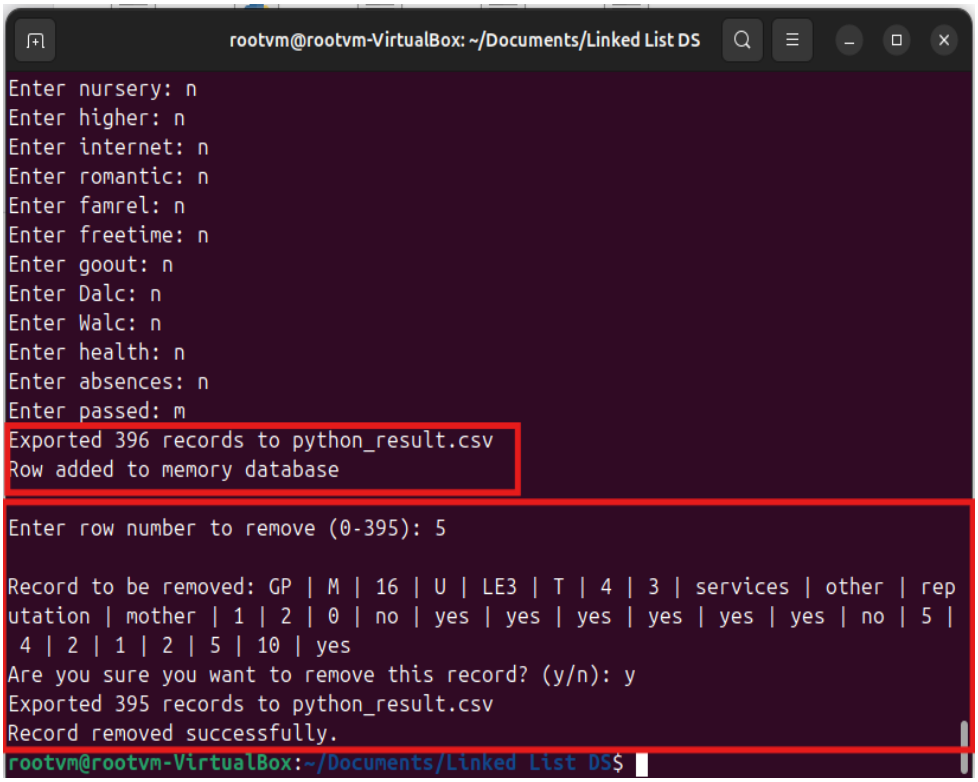


Figure 47 – Removing data from Memory Database using Python

d. Enable monitor_changes() to get real time updations.

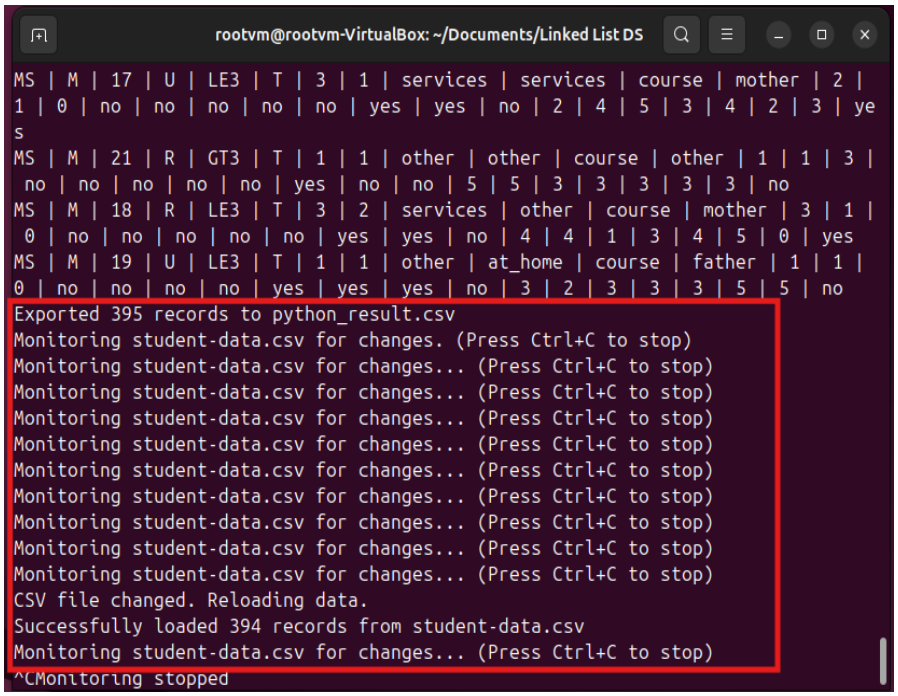


Figure 48 – Monitoring the write operation of Original Dataset using Python

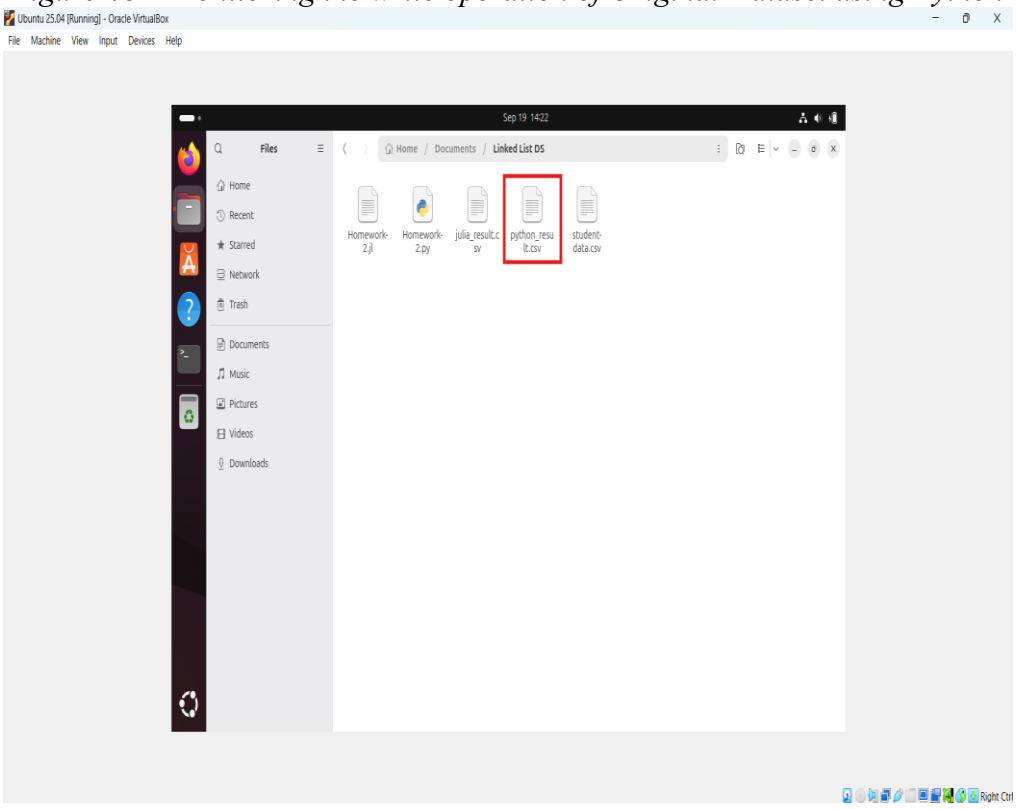


Figure 49 – Resultant file created from Memory Database using Python

e. Implementation of Bubble Sort and Insertion Sort on the records available in memory database to sort the data using any specific key/column.

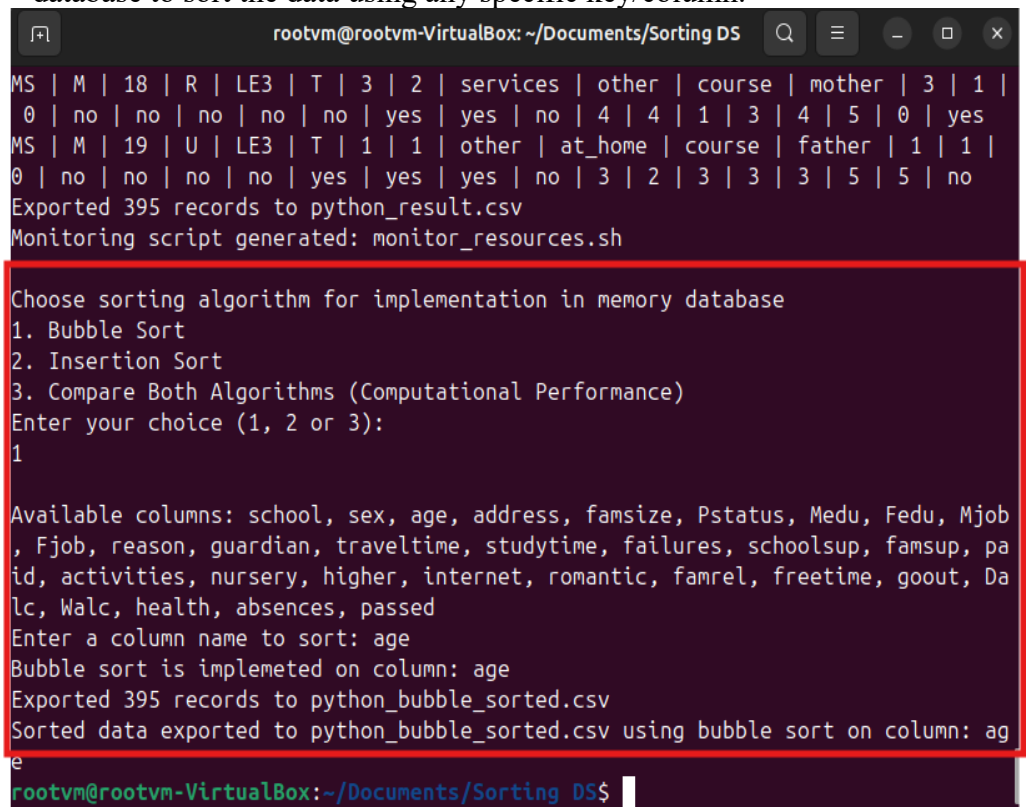


Figure 50 – Organizing records using bubble sort in Python

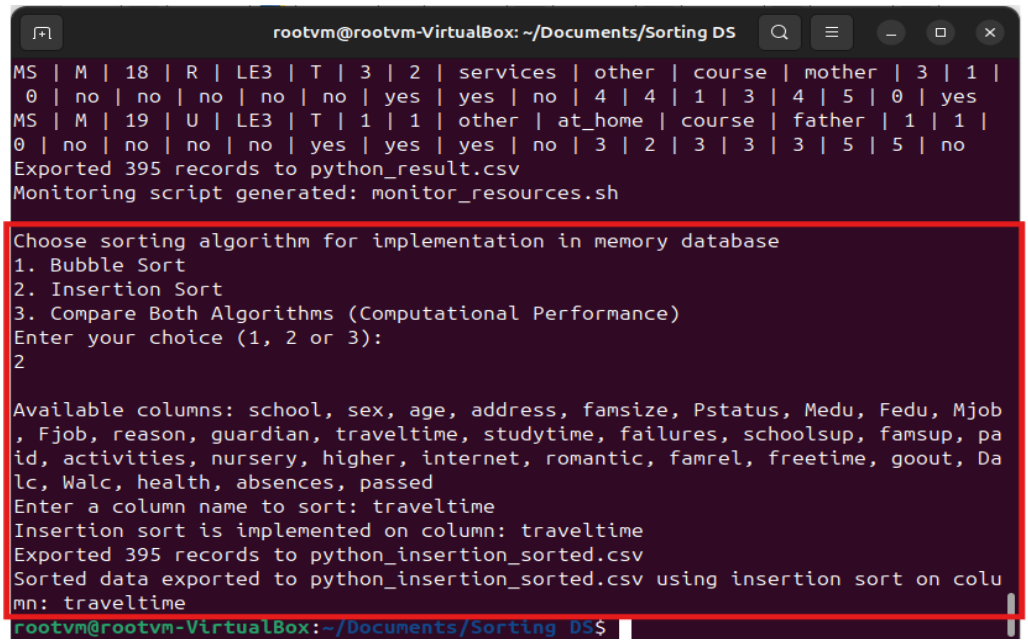


Figure 51 – Organizing records using insertion sort in Python

f. Monitoring system health and comparing the CPU - Disk consumption of both sorting algorithms that can be extracted in the form of logs in .txt format.

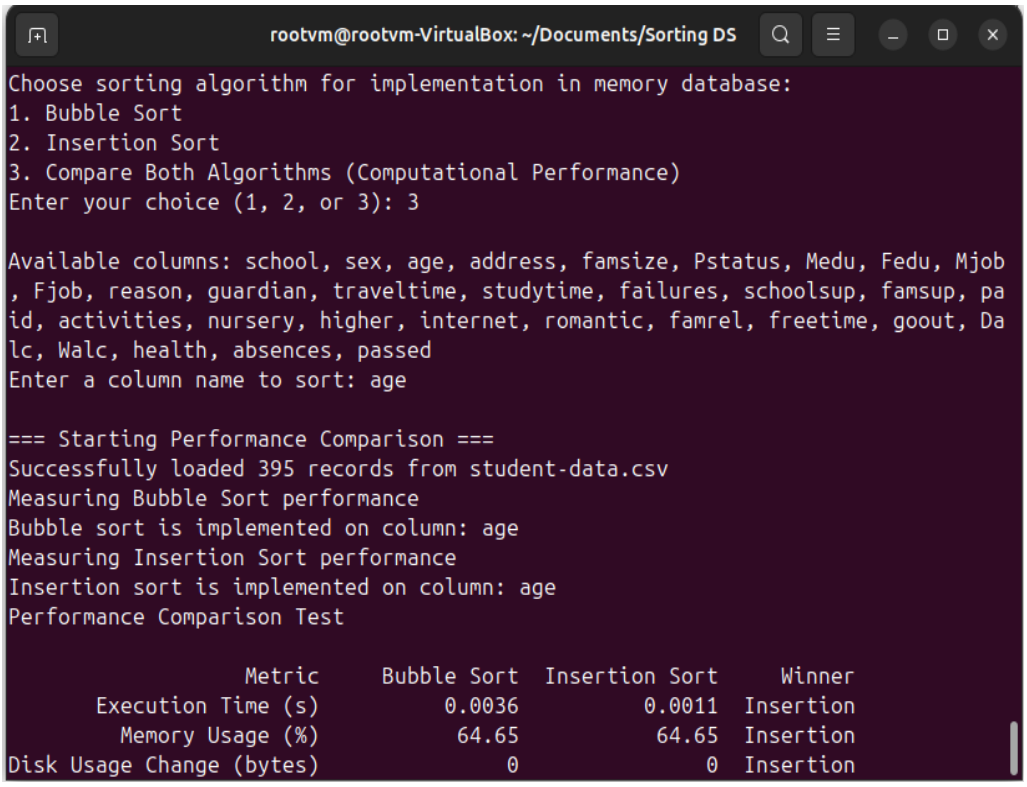


Figure 52 – Comparing CPU/Disk Consumption of both Bubble and Insertion Sort

Executable Code

1. Python

Please refer to the link https://onlinegdb.com/O0Z0eE-qr to access the executable python project.

2. Julia

onlinegdb.com does not support Julia programming language.

GitHub Repository

Link:

https://github.com/devsachink22/Implementation-of-Linked-List-Recursion-Sorting-CPU-Disk-Consumption

Appendix

1. Develop a memory database with singly linked list using Julia programming and load the data from student-data.csv file into memory database When a row is added or removed from student-data.csv, memory database should adapt the changes and export the memory database into resultant csv using recursive function. Moreover, bubble and insertion sort should be further implemented on memory database to sort the data and monitor the CPU as well as disk usage using at least two Linux commands.

```
using CSV
using DataFrames
using Dates
import Statistics

# Performance measurement structure
mutable struct performance_metrics
    algorithm::String
    execution_time::Float64
    cpu_usage::Float64
    memory_usage::Float64
    disk_space_usage::Int64
    timestamp::String
end

# Node definition for singly linked list.
mutable struct Node
    """A node in a singly linked list, storing a dictionary of key-value pairs and a reference to the
next node."""
    data::Dict{String, String}
    next::Union{Node, Nothing}
    function Node(data::Dict{String, String})
```

```
        new(data, nothing)
    end
end

# Implementation of singly linked list.
mutable struct singly_linked_list
    # head refers to the first node
    head::Union{Node, Nothing}
    # tail refers to the last node
    tail::Union{Node, Nothing}
    # size refers to the number of nodes
    size::Int
    function singly_linked_list()
        new(nothing, nothing, 0)
    end
end

# Append a node to the last of the list.
function add_node(list::singly_linked_list, data::Dict{String, String})
    new_node = Node(data)
    if list.head === nothing
        list.head = new_node
        list.tail = new_node
    else
        list.tail.next = new_node
        list.tail = new_node
    end
    list.size += 1
    return new_node
end

# Remove a node at the specific index.
function remove_node(list::singly_linked_list, index::Int)
    if index < 0 || index >= list.size
        return false
    end
    if index == 0
        list.head = list.head.next
        if list.head === nothing
            list.tail = nothing
        end
        list.size -= 1
        return true
    end
    current = list.head
    for _ in 1:(index - 1)
```

```
        current = current.next
    end
    current.next = current.next.next
    if current.next === nothing
        list.tail = current
    end
    list.size -= 1
    return true
end

"""Return the node at the given index (0-based). Returns nothing if the index is invalid."""
function get_node_at(list::singly_linked_list, index::Int)
    if index < 0 || index >= list.size
        return nothing
    end
    current = list.head
    for _ in 1:index
        current = current.next
    end
    return current
end

# Print all the nodes in the linked list.
function display_list(list::singly_linked_list)
    current = list.head
    while current !== nothing
        println(current.data)
        current = current.next
    end
end

# Memory database built on a singly linked list backed by CSV file.
mutable struct memory_database
    csv_file::String
    data_list::singly_linked_list
    last_modified_time::Float64
    headers::Vector{String}
    performance_logs::Vector{performance_metrics}
    function memory_database(csv_file::String)
        new(csv_file, singly_linked_list(), 0.0, String[], performance_metrics[])
    end
end

# Recursive function to load CSV data into linked list.
function load_csv(db::memory_database, rows::Vector{Vector{String}}, index::Int)
    if index > length(rows)
```

```
        return
    end

    row = rows[index]
    # Create dictionary with headers as keys.
    row_data = Dict{String,String}()
    for (i, h) in enumerate(db.headers)
        if i <= length(row)
            row_data[h] = row[i]
        else
            row_data[h] = ""
        end
    end
    add_node(db.data_list, row_data)
    load_csv(db, rows, index + 1)
end

# Recursive function to load data from CSV file.
function load_data(db::memory_database)
    if !isfile(db.csv_file)
        println("CSV file not found: ", db.csv_file)
        return
    end
    rows = CSV.File(db.csv_file; header=false) |> collect
    if isempty(rows)
        println("CSV file is empty")
        return
    end

    # first row = headers
    db.headers = [string(x) for x in rows[1]]
    data_rows = [Vector{String}(string.(row)) for row in rows[2:end]]

    db.data_list = singly_linked_list()
    if !isempty(data_rows)
        load_csv(db, data_rows, 1)
    end

    db.last_modified_time = stat(db.csv_file).mtime
    println("Successfully loaded $(db.data_list.size) records from $(db.csv_file)")
end

# Recursively write linked list nodes to an open CSV file handle.
function export_csv(io::IO, node::Union{Node,Nothing}, headers::Vector{String})
    if node === nothing
        return
```

```
    end
    row = [get(node.data, h, "") for h in headers]
    println(io, join(row, ","))
    export_csv(io, node.next, headers)
end

# Export the linked list to a CSV file with headers and data.
function export_to_csv(db::memory_database, output_file::String="julia_result.csv")
    if isempty(db.headers)
        println("No data to export")
        return
    end
    open(output_file, "w") do io
        println(io, join(db.headers, ","))
        export_csv(io, db.data_list.head, db.headers)
    end
    println("Exported $(db.data_list.size) records to $output_file")
end

# Print the database contents with headers and row values.
function display_data(db::memory_database)
    println("\nCurrent Memory Database Contents:")
    println("Total records: ", db.data_list.size)
    if !isempty(db.headers)
        println(join(db.headers, " | "))
    end
    current = db.data_list.head
    while current !== nothing
        row_data = current.data
        row_values = [get(current.data, h, "") for h in db.headers]
        println(join(row_values, " | "))
        current = current.next
    end
end

# Check for the modification of underlying CSV.
function check_changes(db::memory_database)
    if !isfile(db.csv_file)
        return false
    end
    current_modified = stat(db.csv_file).mtime
    if current_modified > db.last_modified_time
        println("CSV file changed. Reloading data.")
        load_data(db)
        export_to_csv(db, "julia_result.csv")
        return true
```

```
    end
    return false
end

"""Continuously monitor the CSV file for changes at interval seconds. Reloads data if changes
are detected."""
function monitor_changes(db::memory_database, interval::Int)
    println("Monitoring $(db.csv_file) for changes. (Ctrl+C to stop)")
    while true
        check_changes(db)
        sleep(interval)
    end
end

# Prompt the user to enter values for each column header and append a new row to the linked
list.
function add_row(db::memory_database)
    if isempty(db.headers)
        println("No headers available. Load data first.")
        return
    end
    row_data = Dict{String,String}()
    for h in db.headers
        print("Enter $h: ")
        value = readline()
        row_data[h] = value
    end
    add_node(db.data_list, row_data)
    export_to_csv(db, "julia_result.csv")
    println("Row added to memory database")
end

# Prompt the user for a row index and remove that row from the database.
function remove_row(db::memory_database)
    if db.data_list.size == 0
        println("No records to remove.")
        return
    end
    print("Enter row number to remove (0-$(db.data_list.size-1)): ")
    index = parse(Int, readline())
    if index < 0 || index >= db.data_list.size
        println("Invalid row number.")
        return
    end
    row_data = get_node_at(db.data_list, index)
    row_values = [get(row_data.data, header, "") for header in db.headers]
```

```
    println("Record to be removed: ", join(row_values, " | "))
    print("Are you sure you want to remove this record? (y/n): ")
    confirm = lowercase(readline())
    if confirm == "y"
        if remove_node(db.data_list, index)
            export_to_csv(db, "julia_result.csv")
            println("Record removed successfully.")
        end
    end
end
# Recursive function to validate whether the specified column exist in headers or not.
function validate_key(db::memory_database, key::AbstractString)
    if db.headers === nothing || !(String(key) in db.headers)
        println("Error: Column '$key' is not found in CSV headers.")
        if db.headers !== nothing
            println("Available Columns: $(join(db.headers, ", "))")
        end
        return false
    end
    return true
end

# Get current system resource usage
function get_system_statistics()
    try
        # Get memory usage
        memory_info = Sys.free_memory()
        total_memory = Sys.total_memory()
        memory_usage = ((total_memory - memory_info) / total_memory) * 100

        # Get disk usage for current directory
        disk_info = stat(".")
        disk_usage = disk_info.size

        return (memory_usage = round(memory_usage, digits=2), disk_usage = disk_usage)
    catch e
        println("Error getting system stats: $e")
        return (memory_usage = 0.0, disk_usage = 0)
    end
end

# Measure performance of sorting algorithm
function measure_sort_performance(db::memory_database, sort_func::Function, key::String,
sort_name::String)
    println("Measuring $sort_name performance")
```

```
    # Get initial disk usage
    initial_stats = get_system_statistics()
    initial_disk = initial_stats.disk_usage

    # Start time
    start_time = time()

    # Execute the sort function
    sort_func(db, key)

    # End time
    end_time = time()

    # Get final system stats
    final_stats = get_system_statistics()
    final_disk = final_stats.disk_usage

    # Calculate metrics
    execution_time = end_time - start_time

    performance_data = performance_metrics(
        sort_name,
        execution_time,
        initial_stats.memory_usage,  # Using memory as proxy for CPU in this simplified version
        final_stats.memory_usage,
        final_disk - initial_disk,
        string(now())
    )

    push!(db.performance_logs, performance_data)
    return performance_data
end

# Recursive function to implement the bubble sort for the linked list.
function bubble_sort(db::memory_database, key::String)
    if !validate_key(db, String(key))
        return false
    end
    if db.data_list.size <= 1
        println("Not enough data to sort.")
        return true
    end
    # Convert linked list to array for easier sorting
    nodes = Node[]
    current = db.data_list.head
    while current !== nothing
```

```
        push!(nodes, current)
        current = current.next
    end
    # Bubble sort algorithm
    n = length(nodes)
    for i in 1:n-1
        swapped = false
        for j in 1:n-i
            # Handle missing keys by treating them as empty strings
            value1 = get(nodes[j].data, key, "")
            value2 = get(nodes[j+1].data, key, "")
            # Try to convert to numeric for proper comparison
            try
                val1 = isempty(value1) ? 0.0 : parse(Float64, value1)
                val2 = isempty(value2) ? 0.0 : parse(Float64, value2)
                if val1 > val2
                    nodes[j].data, nodes[j+1].data = nodes[j+1].data, nodes[j].data
                    swapped = true
                end
            catch
                # If conversion fails to numeric, compare as strings
                if value1 > value2
                    nodes[j].data, nodes[j+1].data = nodes[j+1].data, nodes[j].data
                    swapped = true
                end
            end
        end
        if !swapped
            break
        end
    end
    println("Bubble sort is implemented on column: $key")
    return true
end

# Recursive function to implement the insertion sort for the linked list.
function insertion_sort(db::memory_database, key::String)
    if !validate_key(db, String(key))
        return false
    end
    if db.data_list.size <= 1
        println("Not enough data to sort.")
        return true
    end
    # Convert to array for sorting as we are using a singly linked list
    nodes = Node[]
```

```
    current = db.data_list.head
    while current !== nothing
        push!(nodes, current)
        current = current.next
    end
    # Insertion sort algorithm
    for i in 2:length(nodes)
        j = i
        while j > 1
            value1 = get(nodes[j-1].data, key, "")
            value2 = get(nodes[j].data, key, "")
            # Try to convert to numeric for proper comparison
            try
                val1 = isempty(value1) ? 0.0 : parse(Float64, value1)
                val2 = isempty(value2) ? 0.0 : parse(Float64, value2)
                if val1 > val2
                    nodes[j-1].data, nodes[j].data = nodes[j].data, nodes[j-1].data
                    j -= 1
                else
                    break
                end
            catch
                # If conversion fails to numeric, compare as strings
                if value1 > value2
                    nodes[j-1].data, nodes[j].data = nodes[j].data, nodes[j-1].data
                    j -= 1
                else
                    break
                end
            end
        end
    end
    println("Insertion sort is implemented on column: $key")
    return true
end

# Recursive function to choose a column from header for implementing the sorting algorithm.
function choose_sort_key(db::memory_database)
    if isempty(db.headers)
        println("No data is available. Please load the data first.")
        return nothing
    end
    println("\nAvailable columns: $(join(db.headers, ", "))")
    print("Enter a column name to sort: ")
    key_input = readline()
    key = string(strip(key_input))  # Convert to String explicitly
```

```
    if !(key in db.headers)
        println("Error: '$key' is not a valid column name.")
        return nothing
    end
    return key
end

# Function that gives the options of sorting algorithm to implement.
function opt_sorting_algorithm()
    println("\nChoose sorting algorithm for implementation in memory database:")
    println("1. Bubble Sort")
    println("2. Insertion Sort")
    println("3. Compare Both Algorithms (Computational Performance)")
    print("Enter your choice (1, 2, or 3): ")
    algorithm_choice = strip(readline())
    if algorithm_choice == "1"
        return "bubble"
    elseif algorithm_choice == "2"
        return "insertion"
    elseif algorithm_choice == "3"
        return "compare"
    else
        println("Invalid choice. Using Bubble Sort as default.")
        return "bubble"
    end
end

# Recursive function to export the sorted data to CSV.
function export_sorted_csv(db::memory_database, output_file::String, sort_type::String,
key::Union{String, Nothing}=nothing)
    if key === nothing
        key = choose_sort_key(db)
        if key === nothing
            return false
        end
    end
    if !validate_key(db, key)
        return false
    end
    if sort_type == "bubble"
        bubble_sort(db, String(key))
    elseif sort_type == "insertion"
        insertion_sort(db, String(key))
    else
        println("Sort type is invalid. Please use bubble/insertion sort.")
```

```
        return
    end
    export_to_csv(db, output_file)
    println("Sorted data exported to $output_file using $sort_type sort on column: $key")
    return true
end

# Compare performance of both sorting algorithms
function compare_sorting_algorithms(db::memory_database)
    if isempty(db.headers)
        println("No data available. Load data first.")
        return
    end

    key = choose_sort_key(db)
    if key === nothing
        return
    end

    println("\nStarting Performance Comparison")

    # Create a copy of the data for fair comparison
    original_nodes = Dict{String, String}[]
    current = db.data_list.head
    while current !== nothing
        push!(original_nodes, copy(current.data))
        current = current.next
    end

    # Test Bubble Sort
    load_data(db)  # Reload original data
    bubble_perf = measure_sort_performance(db, bubble_sort, key, "Bubble Sort")

    # Restore original data for insertion sort test
    db.data_list = singly_linked_list()
    for node_data in original_nodes
        add_node(db.data_list, node_data)
    end

    # Test Insertion Sort
    insertion_perf = measure_sort_performance(db, insertion_sort, key, "Insertion Sort")

    # Display comparison results
    display_performance_comparison(bubble_perf, insertion_perf)

    # Save performance logs to file
```

```
    save_performance_logs(db)
end

# Display performance comparison results
function display_performance_comparison(bubble_perf::performance_metrics,
insertion_perf::performance_metrics)
    println("Performance Comparison Test")

    println("\n$(lpad("Metric", 25)) $(lpad("Bubble Sort", 15)) $(lpad("Insertion Sort", 15))
$(lpad("Winner", 10))")

    # Execution Time
    bubble_time = bubble_perf.execution_time
    insertion_time = insertion_perf.execution_time
    time_winner = bubble_time < insertion_time ? "Bubble" : "Insertion"
    println("$(lpad("Execution Time (s)", 25)) $(lpad(round(bubble_time, digits=4), 15))
$(lpad(round(insertion_time, digits=4), 15)) $(lpad(time_winner, 10))")

    # Memory Usage
    bubble_memory = bubble_perf.cpu_usage
    insertion_memory = insertion_perf.cpu_usage
    memory_winner = abs(bubble_memory) < abs(insertion_memory) ? "Bubble" : "Insertion"
    println("$(lpad("Memory Usage (%)", 25)) $(lpad(round(bubble_memory, digits=2), 15))
$(lpad(round(insertion_memory, digits=2), 15)) $(lpad(memory_winner, 10))")

    # Disk Usage Change
    bubble_disk = bubble_perf.disk_space_usage
    insertion_disk = insertion_perf.disk_space_usage
    disk_winner = bubble_disk < insertion_disk ? "Bubble" : "Insertion"
    println("$(lpad("Disk Usage Change (bytes)", 25)) $(lpad(bubble_disk, 15))
$(lpad(insertion_disk, 15)) $(lpad(disk_winner, 10))")
end

# Save performance logs to CSV file
function save_performance_logs(db::memory_database)
    if isempty(db.performance_logs)
        return
    end

    log_file = "julia_sorting_performance_logs.txt"
    try
        # Create DataFrame from performance logs
        df = DataFrame(
            timestamp = [log.timestamp for log in db.performance_logs],
            algorithm = [log.algorithm for log in db.performance_logs],
            execution_time = [log.execution_time for log in db.performance_logs],
```

```
            memory_usage = [log.cpu_usage for log in db.performance_logs],
            disk_space_usage = [log.disk_space_usage for log in db.performance_logs]
        )
        CSV.write(log_file, df)
        println("\nPerformance logs saved to $log_file")
    catch e
        println("Error saving performance logs: $e")
    end
end

# Generate shell script for system monitoring
function generate_monitoring_script()
    script_content =

    try
        open("julia_monitor_resources.sh", "w") do file
            write(file, script_content)
        end
        # Make the script executable (Unix/Linux/Mac)
        run(`chmod +x julia_monitor_resources.sh`)
        println("Monitoring script generated: julia_monitor_resources.sh")
    catch e
        println("Error generating monitoring script: $e")
    end
end

# Main Execution
function main()
    # Initialize the memory database
    db = memory_database("student-data.csv")

    # Load initial data
    load_data(db)

    # Display loaded data
    display_data(db)

    # Export to new CSV
    export_to_csv(db, "julia_result.csv")

    # Generate monitoring script
    generate_monitoring_script()
    # Ask user to choose a sorting algorithm
    sort_type = opt_sorting_algorithm()
    if sort_type == "bubble"
        # Sort and export the nodes using bubble sort
```

```
        export_sorted_csv(db, "julia_bubble_sorted.csv", "bubble")
    elseif sort_type == "insertion"
        # Sort and export the nodes using insertion sort
        export_sorted_csv(db, "julia_insertion_sorted.csv", "insertion")
    elseif sort_type == "compare"
        # Compare both algorithms
        compare_sorting_algorithms(db)
    end
    # Monitors real time changes in student-data.csv file
    # monitor_changes(db, 5)

    # Insert a new row in result file
    # add_row(db)

    # Remove a particular row from result file
    # remove_row(db)
end
main()
```

2. Develop a memory database with doubly linked list using Python programming and load the data from student-data.csv file into memory database When a row is added or removed from student-data.csv, memory database should adapt the changes and export the memory database into resultant csv using recursive function. Moreover, bubble and insertion sort should be further implemented on memory database to sort the data and monitor the CPU as well as disk usage using at least two Linux commands.

```
import csv
import os
import time
import subprocess
import psutil

"""Node class for doubly linked list."""
class Node:
    def __init__(self, data=None):
        self.data = data
        self.prev = None
        self.next = None

"""Implementation of doubly linked list."""
class doubly_linked_list:
    def __init__(self):
        # head refers to the first node
        self.head = None
```

```
    # tail refers to the last node
    self.tail = None
    # size refers to the number of nodes
    self.size = 0

"""Append a node to the end of the list."""
def add_node(self, data):
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        self.tail = new_node
    else:
        new_node.prev = self.tail
        self.tail.next = new_node
        self.tail = new_node
    self.size += 1
    return new_node

"""Return the node at the given index (0-based). Returns nothing if the index is invalid."""
def remove_node(self, index):
    if index < 0 or index >= self.size:
        return False

    current = self.head
    for i in range(index):
        current = current.next

    if current.prev:
        current.prev.next = current.next
    else:
        self.head = current.next

    if current.next:
        current.next.prev = current.prev
    else:
        self.tail = current.prev

    self.size -= 1
    return True

"""Print all the nodes in the linked list."""
def display_list(self):
    current = self.head
    while current:
        print(current.data)
        current = current.next
```

```
# Memory database built on a doubly linked list backed by CSV file.
class memory_database:
    def __init__(self, csv_file):
        self.csv_file = csv_file
        self.data_file = doubly_linked_list()
        self.last_modified_time = 0
        self.headers = None
        self.performance_logs = []

    """Recursive function to load CSV data into linked list."""
    def load_csv(self, rows, headers, index=0):
        if index >= len(rows):
            return

        # Create dictionary with headers as keys
        row_data={}
        for i, header in enumerate(headers):
            if i < len(rows[index]):
                row_data[header] = rows[index][i]
            else:
                row_data[header] = ""  # Handle missing values
        self.data_file.add_node(row_data)
        self.load_csv(rows, headers, index + 1)

    """Recursive function to load data from CSV file."""
    def load_data(self):
        try:
            with open(self.csv_file, 'r', newline='', encoding='utf-8') as file:
                reader = csv.reader(file)
                rows = list(reader)

                if not rows:
                    print("CSV file is empty")
                    return

                # first row = headers
                self.headers = rows[0]

                # Clear existing data
                self.data_file = doubly_linked_list()

                # Load data recursively
                if rows:
                    self.load_csv(rows[1:], self.headers)  # Skip header
```

```
            # Update last modified time
            self.last_modified_time = os.path.getmtime(self.csv_file)

            print(f"Successfully loaded {self.data_file.size} records from {self.csv_file}")

    except FileNotFoundError:
        print("CSV file not found")
    except Exception as e:
        print(f"Error loading CSV: {e}")

"""Recursively write linked list nodes to an open CSV file handle."""
def export_csv(self, writer, node):
    if node is None:
        return
    row = [node.data.get(header, "") for header in self.headers]
    writer.writerow(row)
    self.export_csv(writer, node.next)

"""Export the linked list to a CSV file with headers and data."""
def export_to_csv(self, output_file):
    if output_file is None:
        output_file = self.csv_file

    if self.headers is None:
        print("No data to export")
        return

    try:
        with open(output_file, 'w', newline='', encoding='utf-8') as file:
            writer = csv.writer(file)
            # Defined headers of python_result.csv file
            writer.writerow(self.headers)

            # Export data to python_result.csv file
            self.export_csv(writer, self.data_file.head)

        print(f"Exported {self.data_file.size} records to {output_file}")

    except Exception as e:
        print(f"Error exporting CSV: {e}")

"""Print the database contents with headers and row values."""
def display_data(self):
    print("\nCurrent Memory Database Contents:")
    print(f"Total records: {self.data_file.size}")
```

```
        if self.headers:
            print(" | ".join(self.headers))

        current = self.data_file.head
        while current:
            row_data = current.data
            row_values = [str(row_data.get(header, "")) for header in self.headers]
            print(" | ".join(row_values))
            current = current.next

    """Check for the modification of underlying CSV."""
    def check_changes(self):
        print(f"Monitoring {self.csv_file} for changes... (Press Ctrl+C to stop)")
        while True:
            try:
                current_modified = os.path.getmtime(self.csv_file)
                if current_modified > self.last_modified_time:
                    print("CSV file changed. Reloading data.")
                    self.load_data()
                    return True
            except:
                pass
            return False

    """Continuously monitor the CSV file for changes at interval seconds. Reloads data if changes
are detected."""
    def monitor_changes(self, interval=5):
        print(f"Monitoring {self.csv_file} for changes. (Press Ctrl+C to stop)")
        while True:
            try:
                self.check_changes()
                time.sleep(interval)
            except KeyboardInterrupt:
                print("Monitoring stopped")
                break
    """Prompt the user to enter values for each column header and append a new row to the linked
list."""
    def add_row(self, row_data):
        if self.headers is None:
            print("No headers available. Load data first.")
            return

        newRow = []
        for i, header in enumerate(self.headers):
            value = input(f"Enter {header}: ")
            row_data.append(value)
```

```
        # Convert list to dictionary using headers
        dict_data = {}
        for i, header in enumerate(self.headers):
            if i < len(row_data):
                dict_data[header] = row_data[i]
            else:
                dict_data[header] = ""

    self.data_file.add_node(dict_data)
    self.export_to_csv("python_result.csv")
    print("Row added to memory database")

"""Prompt the user for a row index and remove that row from the database."""
def remove_row(self, index):
    if self.data_file.size == 0:
        print("No records to remove.")
        return

    try:
        index = int(input(f"\nEnter row number to remove (0-{self.data_file.size-1}): "))
        if 0 <= index < self.data_file.size:
            # Show what will be removed
            current = self.data_file.head
            for i in range(index):
                current = current.next
            row_data = current.data
            row_values = [str(row_data.get(header, "")) for header in self.headers]
            print(f"\nRecord to be removed: {' | '.join(row_values)}")

            confirm = input("Are you sure you want to remove this record? (y/n): ").lower()
            if confirm == 'y':
                if self.data_file.remove_node(index):
                    self.export_to_csv("python_result.csv")
                    print("Record removed successfully.")
        else:
            print("Invalid row number.")
    except ValueError:
        print("Please enter a valid number.")

"""Recursive function to validate whether the specified column exist in headers or not."""
def validate_key(self, key):
    if key not in self.headers:
        print(f'Error: Column '{key}' is not found in CSV headers.")
        print(f"Available Columns: {', '.join(self.headers)}")
        return False
```

```
    return True

"""Get current system resource usage"""
def get_system_statistics(self):
    try:
        # CPU usage
        cpu_percent = psutil.cpu_percent(interval=0.1)

        # Memory usage
        memory = psutil.virtual_memory()
        memory_usage = memory.percent

        # Disk usage for current directory
        disk = psutil.disk_usage('.')
        disk_usage = disk.percent

        return {
            'cpu_percent': cpu_percent,
            'memory_percent': memory_usage,
            'disk_percent': disk_usage,
            'timestamp': time.time()
        }
    except Exception as e:
        print(f"Error getting system stats: {e}")
        return None

"""Run shell command and return output"""
def run_shell_command(self, command):
    try:
        result = subprocess.run(command, shell=True, capture_output=True, text=True)
        return result.stdout.strip()
    except Exception as e:
        print(f"Error running command {command}: {e}")
        return ""

"""Measure performance of sorting algorithm"""
def measure_sort_performance(self, sort_func, key, sort_name):
    print(f"\nMeasuring {sort_name} performance")

    # Get initial disk usage
    initial_disk = self.run_shell_command("du -sb . | cut -f1")

    # Get initial system stats
    initial_stats = self.get_system_statistics()

    # Start time
```

```
        start_time = time.time()

        # Execute the sort function
        sort_func(key)

        # End time
        end_time = time.time()

        # Get final system stats
        final_stats = self.get_system_statistics()

        # Get final disk usage
        final_disk = self.run_shell_command("du -sb . | cut -f1")

        # Calculate metrics
        execution_time = end_time - start_time

        performance_data = {
            'algorithm': sort_name,
            'execution_time': execution_time,
            'initial_disk_bytes': int(initial_disk) if initial_disk else 0,
            'final_disk_bytes': int(final_disk) if final_disk else 0,
            'disk_usage_change': (int(final_disk) - int(initial_disk)) if initial_disk and final_disk else
0,
            'initial_cpu': initial_stats['cpu_percent'] if initial_stats else 0,
            'final_cpu': final_stats['cpu_percent'] if final_stats else 0,
            'cpu_usage_change': (final_stats['cpu_percent'] - initial_stats['cpu_percent']) if
initial_stats and final_stats else 0,
            'timestamp': time.strftime('%Y-%m-%d %H:%M:%S')
        }

        self.performance_logs.append(performance_data)
        return performance_data

    """Recursive function to implement the bubble sort for the linked list."""
    def bubble_sort(self, key):
        if not self.validate_key(key):
            return False
        if self.data_file.size <= 1:
            print("Not enough data to sort.")
            return True
        # Convert linked list to array for easier sorting
        nodes = []
        current = self.data_file.head
        while current:
            nodes.append(current)
```

```
        current = current.next
    # Bubble sort algorithm
    n = len(nodes)
    for i in range(0,n-1):
        swapped = False
        for j in range(0, n-i-1):
            # Handle missing keys by treating them as empty strings
            value1 = nodes[j].data.get(key, "")
            value2 = nodes[j+1].data.get(key, "")
            # Try to convert to numeric for proper comparison
            try:
                value1 = float(value1) if value1 else 0
                value2 = float(value2) if value2 else 0
            except ValueError:
                # If conversion fails to numeric, compare as strings
                pass
            if value1 > value2:
                # Swap nodes by swapping their data
                nodes[j].data, nodes[j+1].data = nodes[j+1].data, nodes[j].data
                swapped = True
        if not swapped:
            break
    print(f"Bubble sort is implemeted on column: {key}")

"""Recursive function to implement the insertion sort for the linked list."""
def insertion_sort(self, key):
    if not self.validate_key(key):
        return False
    if self.data_file.size <= 1:
        print("Not enough data to sort.")
        return True
    current = self.data_file.head  # Start from first node
    while current:
        current_data = current.data
        # Identify the correct position for current node
        prev_node = current.prev
        while prev_node is not None:
            # Handle missing keys by treating them as empty strings
            prev_value = prev_node.data.get(key, "")
            current_value = current_data.get(key, "")
            # Try to convert to numeric for proper comparison
            try:
                prev_value = float(prev_value) if prev_value else 0
                current_value = float(current_value) if current_value else 0
            except ValueError:
                # If conversion fails to numeric, compare as strings
```

```
                pass
            if prev_value > current_value:
                # Shift node to the right
                prev_node.next.data, prev_node.data = prev_node.data, prev_node.next.data
                prev_node = prev_node.prev
            else:
                break
        if prev_node is None:
            # Update head if required
            self.data_file.head.data, current.data = current.data, self.data_file.head.data
        current = current.next
    print(f"Insertion sort is implemented on column: {key}")

"""Recursive function to opt a column from header for implementing the sorting algorithm."""
def choose_sort_key(self):
    if not self.headers:
        print("No data is available. Please load the data first.")
        return None
    print(f"\nAvailable columns: {', '.join(self.headers)}")
    key = input("Enter a column name to sort: ").strip()
    if key not in self.headers:
        print(f"Error: '{key}' is not a valid column name.")
        return None
    return key

"""Function that gives the options of sorting algorithm to implement."""
def opt_sorting_algorithm(self):
    print("\nChoose sorting algorithm for implementation in memory database")
    print("1. Bubble Sort")
    print("2. Insertion Sort")
    print("3. Compare Both Algorithms (Computational Performance)")
    print("Enter your choice (1, 2 or 3): ")
    algorithm_choice = input().strip()
    if algorithm_choice == "1":
        return "bubble"
    elif algorithm_choice == "2":
        return "insertion"
    elif algorithm_choice == "3":
        return "compare"
    else:
        print("Invalid choice. Using Bubble Sort as default.")
        return "bubble"

"""Recursive function to export the sorted data to CSV."""
def export_sorted_csv(self, output_file, sort_type, key=None):
    if key is None:
```

```
            key = self.choose_sort_key()
            if key is None:
                return False
        if not self.validate_key(key):
            return False
        if sort_type == "bubble":
            self.bubble_sort(key)
        elif sort_type == "insertion":
            self.insertion_sort(key)
        else:
            print("Sort type is invalid. Please use bubble/insertion sort.")
            return
        self.export_to_csv(output_file)
        print(f"Sorted data exported to {output_file} using {sort_type} sort on column: {key}")

    """Compare performance of both sorting algorithms"""
    def compare_sorting_algorithms(self):
        if not self.headers:
            print("No data available. Load data first.")
            return

        key = self.choose_sort_key()
        if key is None:
            return

        print("\n=== Starting Performance Comparison ===")

        # Create a copy of the data for fair comparison
        original_nodes = []
        current = self.data_file.head
        while current:
            original_nodes.append(current.data.copy())
            current = current.next

        # Test Bubble Sort
        self.load_data()  # Reload original data
        bubble_perf = self.measure_sort_performance(self.bubble_sort, key, "Bubble Sort")

        # Restore original data for insertion sort test
        self.data_file = doubly_linked_list()
        for node_data in original_nodes:
            self.data_file.add_node(node_data)

        # Test Insertion Sort
        insertion_perf = self.measure_sort_performance(self.insertion_sort, key, "Insertion Sort")
```

```
        # Display comparison results
        self.display_performance_comparison(bubble_perf, insertion_perf)

        # Save performance logs to file
        self.save_performance_logs()

    """Display performance comparison results"""
    def display_performance_comparison(self, bubble_perf, insertion_perf):
        print("Performance Comparison Result")

        print(f"\n{'Metric':<25} {'Bubble Sort':<15} {'Insertion Sort':<15} {'Winner':<10}")

        # Execution Time
        bubble_time = bubble_perf['execution_time']
        insertion_time = insertion_perf['execution_time']
        time_winner = "Bubble" if bubble_time < insertion_time else "Insertion"
        print(f"{'Execution Time (s)':<25} {bubble_time:<15.4f} {insertion_time:<15.4f}
{time_winner:<10}")

        # CPU Usage Change
        bubble_cpu = bubble_perf['cpu_usage_change']
        insertion_cpu = insertion_perf['cpu_usage_change']
        cpu_winner = "Bubble" if abs(bubble_cpu) < abs(insertion_cpu) else "Insertion"
        print(f"{'CPU Usage Change (%)':<25} {bubble_cpu:<15.2f} {insertion_cpu:<15.2f}
{cpu_winner:<10}")

        # Disk Usage Change
        bubble_disk = bubble_perf['disk_usage_change']
        insertion_disk = insertion_perf['disk_usage_change']
        disk_winner = "Bubble" if bubble_disk < insertion_disk else "Insertion"
        print(f"{'Disk Usage Change (bytes)':<25} {bubble_disk:<15} {insertion_disk:<15}
{disk_winner:<10}")

    """Save performance logs to CSV file"""
    def save_performance_logs(self):
        if not self.performance_logs:
            return

        log_file = "python_sorting_performance_logs.txt"
        fieldnames = ['timestamp', 'algorithm', 'execution_time', 'initial_disk_bytes',
                      'final_disk_bytes', 'disk_usage_change', 'initial_cpu', 'final_cpu',
'cpu_usage_change']

        try:
            file_exists = os.path.isfile(log_file)
            with open(log_file, 'a', newline='', encoding='utf-8') as file:
```

```
            writer = csv.DictWriter(file, fieldnames=fieldnames)
            if not file_exists:
                writer.writeheader()
            writer.writerows(self.performance_logs)
        print(f"\nPerformance logs saved to {log_file}")
    except Exception as e:
        print(f"Error saving performance logs: {e}")

"""Generate shell script for system monitoring"""
def generate_monitoring_script(self):
    script_content = ""

    try:
        with open("monitor_resources.sh", "w") as f:
            f.write(script_content)
        os.chmod("monitor_resources.sh", 0o755)
        print("Monitoring script generated: monitor_resources.sh")
    except Exception as e:
        print(f"Error generating monitoring script: {e}")

# Main Execution
if __name__ == "__main__":
    # Initialize the memory database
    db = memory_database("student-data.csv")

    # Load initial data
    db.load_data()

    # Display loaded data
    db.display_data()

    # Export to new CSV
    db.export_to_csv("python_result.csv")

    # Generate monitoring script
    db.generate_monitoring_script()

    # Ask user to choose a sorting algorithm
    opt_type = db.opt_sorting_algorithm()
    if opt_type == "bubble":
        # Sort and export the nodes using bubble sort
        db.export_sorted_csv("python_bubble_sorted.csv", "bubble")
    elif opt_type == "insertion":
        # Sort and export the nodes using insertion sort
        db.export_sorted_csv("python_insertion_sorted.csv", "insertion")
    elif opt_type == "compare":
```

```
    # Compare both algorithms
    db.compare_sorting_algorithms()

# Monitors real time changes in student-data.csv file.
# db.monitor_changes()

# Insert a new row in result file.
# db.add_row([])

# Remove a particular row from result file.
# db.remove_row([])
```